

# TEXT ANALYTICS

## Machine Learning for Text Analytics - Part 1

---

*Francesco Spinnato*

\* [francesco.spinnato@unipi.it](mailto:francesco.spinnato@unipi.it)

University of Pisa



# From Representations to Learning

We learned to turn every document into a **vector**: a document  $d$  becomes  $\mathbf{x} \in \mathbb{R}^M$ , with entries that can contain different information:

- hand-crafted linguistic features
- the frequency of each word
- TF-IDF weights.
- etc...

A collection of  $N$  documents is then a matrix  $\mathbf{X}$  with  $N$  rows and  $M$  columns.

**Machine learning** looks for structure in  $\mathbf{X}$ .

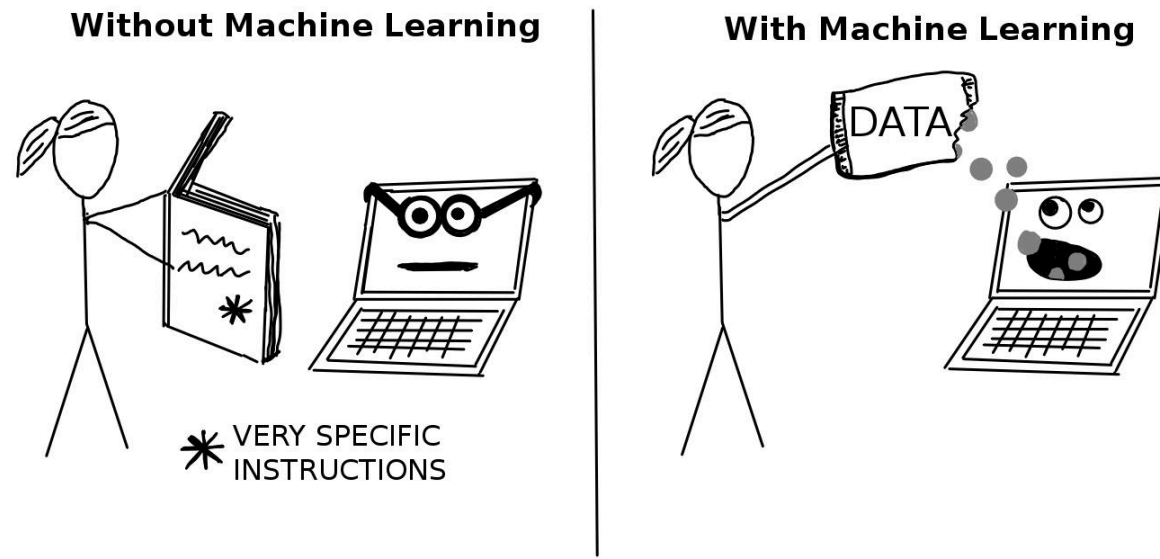
---

The previous lecture wrote  $|F|$  for the number of features. From here on we use the generic machine learning notation:  $M$  features,  $N$  documents.

# Learning from Data

Learning from data is a central idea in Machine Learning.

- A machine learning model is a parameterized function that represents a relationship between inputs and outputs.
- A learning algorithm **adjusts the parameters of the model** using data.



There are two phases when using machine learning models

## **Learning (training)**

- The model is fit to data (training set) by an optimization procedure.
- In this phase the model changes, i.e., its parameters are learned.

## **Inference (prediction)**

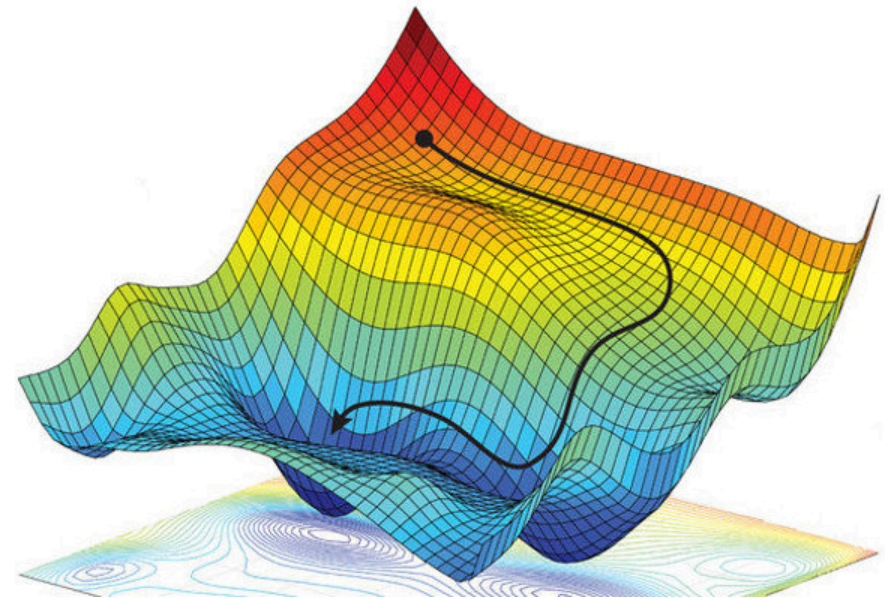
- The trained model is applied to (usually) new data.
- In this phase the model is fixed and does not change.

# Loss function

To learn a model, we define a **loss function**  $\mathcal{L}$  that measures how well (or not) the model is doing.

Learning consists in finding parameters that minimize the loss on the training data.

The choice of loss function depends on the task.



---

We write  $\ell$  for the loss on a single example, and  $\mathcal{L}$  for its average over the training set.

# Supervised and Unsupervised Learning

**Machine learning** looks for structure in the input data  $\mathbf{X}$ . Which structure we can look for depends on one question:

*does each document come with a label?*

Yes

## Supervised

In supervised learning, each training example  $\mathbf{x}$  is associated with a ground-truth label (or target)  $y$ .

No

## Unsupervised

In unsupervised learning, no ground-truth labels are provided.

## Supervised

- **classification**
  - sentiment analysis
  - emotion prediction
  - author attribution
  - spam detection
  - next token prediction
  - language identification
- **regression**
  - rating prediction
  - readability assessment
  - document dating

## Unsupervised

- clustering
- topic modeling
- data generation
- dimensionality reduction

# Supervised Learning in Text Analytics

---

## Learning (training)

Given training data  $(\mathbf{X}, \mathbf{y})$ , the learning algorithm produces a trained model,  $f_{\hat{\theta}}$ :

$$\hat{\theta} = \operatorname{argmin}_{\theta} \mathcal{L}(\theta; \mathbf{X}, \mathbf{y})$$

where  $\mathcal{L}$  is a loss function measuring how well the model is doing.

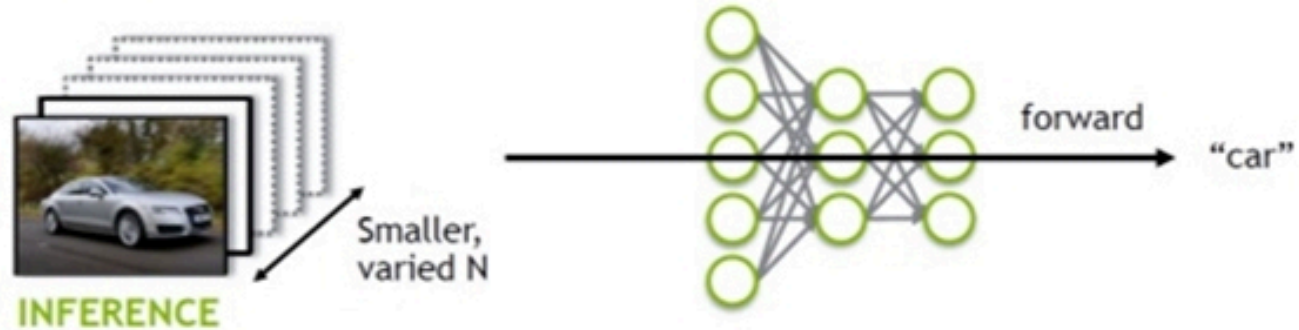
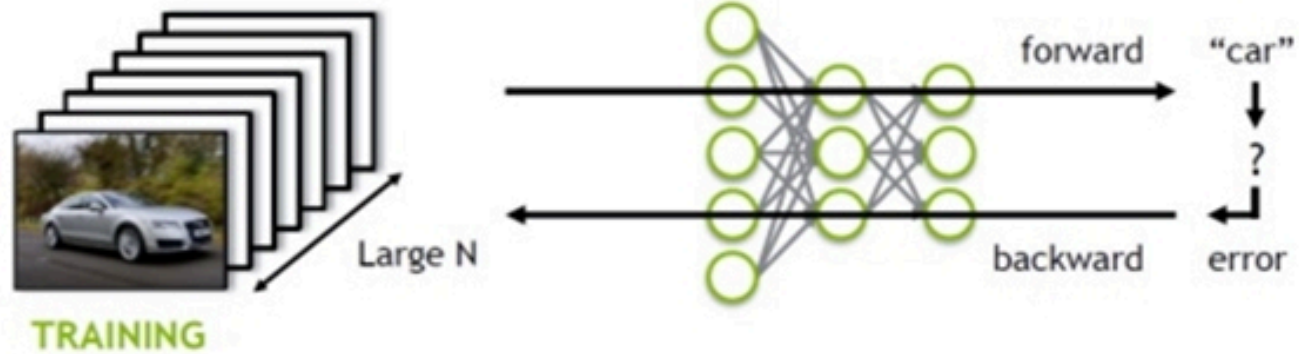
## Inference (prediction)

The trained model is applied to a (possibly new) input:

$$\hat{y} = f_{\hat{\theta}}(\mathbf{x})$$

# Learning and Inference (Supervised)

## TRAINING VS INFERENCE



# Classification

The aim of classification is to predict a **categorical** variable for each document in a dataset.

$$f(\mathbf{X}) = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)] = [\hat{y}_1, \dots, \hat{y}_N] = \hat{\mathbf{y}}$$
$$\hat{\mathbf{y}} \in Y^N$$

where  $Y$  is the set of **classes**:  $y$  is the label of a document,  $c$  one particular class. If  $|Y| = 2$  it is **binary** classification, otherwise it is **multiclass** classification.

- **Binary**: if  $f$  predicts whether a review is positive or negative.
- **Multiclass**: if  $f$  detects the emotion of a post:  $f(\text{post}_1) = \text{anger}$

# From probabilities to classes

Some classification models do not directly output a class label, but **class probabilities**. For a single document:

$$f(\mathbf{x}) = [P(c_1 \mid \mathbf{x}), \dots, P(c_{|Y|} \mid \mathbf{x})], \quad \sum_{c \in Y} P(c \mid \mathbf{x}) = 1$$

Example: for a review, the model outputs probabilities:  $\left[ \underbrace{0.1}_{\text{negative}}, \underbrace{0.9}_{\text{positive}} \right]$

The final predicted class is obtained by:

$$\hat{y} = \operatorname{argmax}_{c \in Y} P(c \mid \mathbf{x})$$

# Document Classification

Assigning documents to categories is the oldest application of text classification:

- **newswire indexing:** the Reuters-21578 collection of news articles was for years the standard benchmark
- **subject classification:** library catalogues, scientific papers, patents
- **routing:** support tickets to the right team, emails to the right folder
- **coding:** medical records to diagnosis codes, legal documents to case types

Every piece of the course so far meets in one pipeline:



# Spam Detection

A binary task: assign each email to **spam** or **not spam** (often called *ham*). It is Naive Bayes' classic success: Bayesian spam filters became widespread in the late 1990s, early 2000s.

The features go **beyond the bag of words**:

- words and phrases: *free, click here, winner*
- form: a high proportion of CAPITALS, many exclamation marks, many links
- metadata: the sender, whether it is in the address book

What makes spam harder than it looks:

- **it is adversarial**: spammers adapt to the filter, *v1agra, fr3e*, hidden text
- **it drifts**: this year's spam does not look like last year's
- **the errors are not equal**: a legitimate email lost in the spam folder costs far more than one spam in the inbox, so **precision** on the spam class matters most

# Sentiment, Emotions, and Affective Dimensions

Beyond syntax and frequency, text often conveys **emotional and attitudinal** information. Capturing these affective aspects adds a semantic layer useful for understanding tone, opinion, or attitude.

Three main affective dimensions:

- **Sentiment polarity:** the overall evaluative orientation of a text.
- **Emotion recognition:** identifying discrete emotional states.
- **Valence-Arousal-Dominance (VAD) model:** representing affect on continuous psychological scales.

# Sentiment Polarity

Measures whether a text expresses a **positive**, **negative** or **neutral** attitude.

- *"This new update is amazing!"* → Positive
- *"The service is terrible."* → Negative
- *"The pen is on the table"* → Neutral

Polarity can be derived:

- from **lexicons**, lists of positive and negative words: the General Inquirer (1966), the MPQA Subjectivity lexicon, the Hu and Liu (2004) lexicon
- from **machine-learning classifiers** trained on labeled data: Naive Bayes and logistic regression, as in this lecture

**Applications:** product reviews, social media monitoring, political stance analysis, customer feedback and brand perception.

Goes beyond polarity and captures **specific emotions** such as joy, anger, fear, sadness, disgust, surprise.

Can be:

- **lexicon-based**, e.g. the NRC Emotion Lexicon (EmoLex), which marks around 14,000 words with the emotions they are associated with
- **model-based**: classifiers trained on texts labeled with emotions

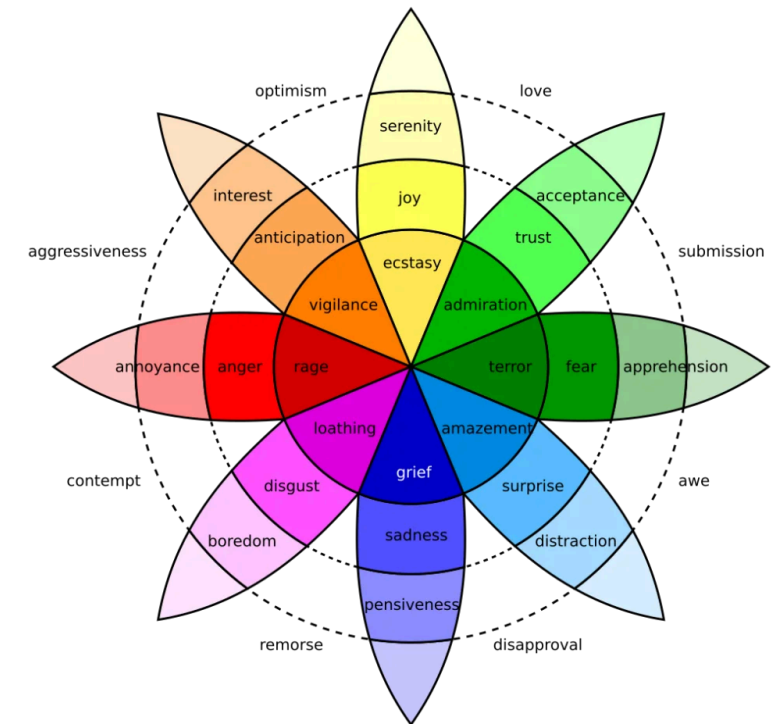
*Examples:*

- *"I can't believe they won!"* → Surprise / Joy
- *"I'm really scared about the results."* → Fear

# Emotions

Two main families of psychological models:

- **Basic emotions**, a fixed set of atomic emotions from which the others derive.
  - **Ekman**: six emotions, proposed as universal across cultures: surprise, happiness, anger, fear, disgust, sadness.
  - **Plutchik's wheel**: eight basic emotions in four opposing pairs: joy-sadness, anger-fear, trust-disgust, anticipation-surprise.
- **Dimensional models**, where an emotion is a point in a space of 2 or 3 dimensions: the VAD model, next.



Datasets have since grown more fine-grained: **GoEmotions** labels Reddit comments with 27 emotion categories plus neutral.

# Valence-Arousal-Dominance (VAD) Model

A **continuous** representation of affective meaning, from psychology and affective computing. Three dimensions:

- **Valence:** the pleasantness of the stimulus, e.g. *sad*  $\leftrightarrow$  *happy* (valence alone is often used directly as a sentiment score.)
- **Arousal:** the level of alertness or energy it provokes, e.g. *calm*  $\leftrightarrow$  *excited*
- **Dominance:** the degree of control it exerts, e.g. *weak*  $\leftrightarrow$  *strong*

Lexicons assign every word a score on each dimension. The NRC VAD lexicon covers 20,000 words, on a 0-1 scale:

| Valence           |      | Arousal        |      | Dominance        |      |
|-------------------|------|----------------|------|------------------|------|
| <i>delightful</i> | .918 | <i>enraged</i> | .962 | <i>powerful</i>  | .991 |
| <i>vacation</i>   | .840 | <i>party</i>   | .840 | <i>authority</i> | .935 |
| <i>torture</i>    | .115 | <i>napping</i> | .046 | <i>weak</i>      | .045 |

# Affect as Feature vs Affect as Target

Affective information can play **two different roles** in ML pipelines.

- **As a feature:** additional signal describing how emotionally charged a text is.
  - e.g. a model predicting persuasive intent, user stance, or engagement may include a sentiment score or an emotion intensity among its input features.
- **As the target of a model:** the goal is to detect or predict affective categories directly from text:
  - *sentiment analysis*: detecting positive, negative or neutral polarity
  - *emotion detection*: identifying discrete emotions such as joy, anger, fear or sadness

Unlike categorical sentiment or emotion tasks, **VAD** is usually employed as a *representational framework* rather than as a direct prediction goal, since it describes affective meaning in a continuous three-dimensional space.

The aim of regression is to predict a **continuous** variable for each document in a dataset.

$$f(\mathbf{X}) = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)] = \hat{\mathbf{y}}$$
$$\hat{\mathbf{y}} \in \mathbb{R}^N$$

# Regression: Predicting a Review's Rating

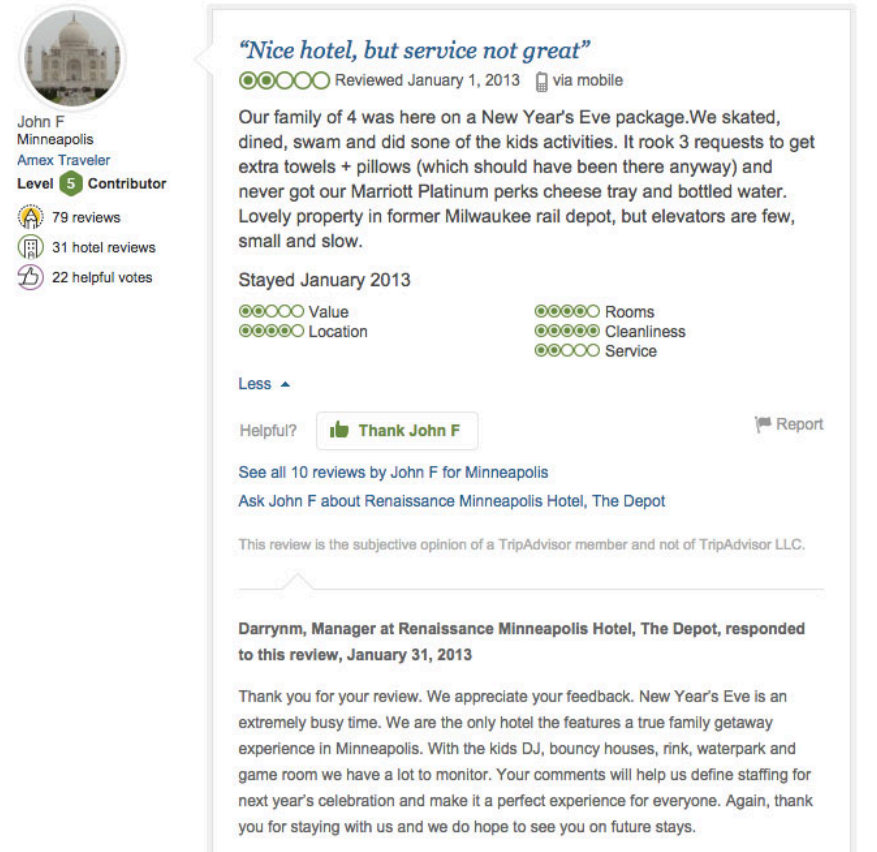
**Input:** the text of a review, as a TF-IDF vector  $\mathbf{x}$ .

**Output:** the number of stars,  $\hat{y} \in \mathbb{R}$ .

Two things to watch:

- the prediction is **unbounded**, so it has to be clipped to  $[1, 5]$ ;
- and stars are **ordinal**, so this could also be framed as 5-class classification.

Regression knows that predicting 1 star instead of 5 is worse than predicting 4; classification treats both as simply wrong.



# Regression: Assessing Readability

- **Input:** two hand-crafted features of a text.
- **Output:** the US school grade needed to understand it,  $\hat{y} \in \mathbb{R}$ .

The **Flesch-Kincaid Grade Level** is a linear regression, fit on reading-test scores:

$$\hat{y} = 0.39 \cdot \underbrace{\frac{\# \text{words}}{\# \text{sentences}}}_{x_1} + 11.8 \cdot \underbrace{\frac{\# \text{syllables}}{\# \text{words}}}_{x_2} - 15.59$$

| Text                                                        | $x_1$ | $x_2$ | $\hat{y}$ |
|-------------------------------------------------------------|-------|-------|-----------|
| <i>The cat sat on the mat. It was happy.</i>                | 4.5   | 1.11  | −0.7      |
| <i>Regularization discourages large coefficient values.</i> | 5     | 3.4   | 26.5      |

- Again the output is **unbounded**: grade −0.7 and grade 26.5 do not exist.

# Supervised Learning in Text Analytics

---

## Naive Bayes

# Bayesian Classifier

Bayes' theorem, with a **class** as the hypothesis and a **document** as the evidence, defines a **probabilistic classifier**:

$$P(\text{class} \mid \text{document}) = \frac{P(\text{document} \mid \text{class}) \times P(\text{class})}{P(\text{document})}$$

| Term                                   | Name              | Meaning                                                     |
|----------------------------------------|-------------------|-------------------------------------------------------------|
| $P(\text{class} \mid \text{document})$ | <b>posterior</b>  | how likely the class is <b>after</b> observing the document |
| $P(\text{document} \mid \text{class})$ | <b>likelihood</b> | how likely this document is, if the class is true           |
| $P(\text{class})$                      | <b>prior</b>      | how likely the class is <b>before</b> seeing any document   |
| $P(\text{document})$                   | <b>evidence</b>   | how likely the document is overall, across all classes      |

# Bayesian Classifier

To pick the most likely class, we only need to **compare** the posteriors.

- For a fixed document,  $P(\text{document})$  is the same denominator for every candidate class,
- so it does not affect which class attains the maximum and can be omitted.

$$P(\text{class} \mid \text{document}) \propto P(\text{document} \mid \text{class}) \times P(\text{class})$$

$$\hat{y} = \operatorname{argmax}_{c \in Y} P(d \mid c) P(c)$$

Both factors can be estimated on a **training set of labeled documents**.

# Bayesian Classifier (Prior): Example

We want to label a news article as **cooking** or **politics**. From a training set of labeled news:

$$P(\text{cooking}) = \frac{\#(\text{news about cooking})}{\#(\text{all news})} = 0.01 \quad P(\text{politics}) = \frac{\#(\text{news about politics})}{\#(\text{all news})} = 0.12$$

These are the **prior beliefs**. Knowing nothing about the article, *politics* is twelve times more likely to be right.

Now we are told the article contains the word *pepper*.

# Bayesian Classifier (Likelihood): Example

$$P(\textit{pepper} \mid \text{cooking}) = \frac{\text{count}(\textit{pepper}, \text{cooking})}{\#(\text{tokens in cooking news})} = 0.001 \quad P(\textit{pepper} \mid \text{politics}) = 0.00002$$

Multiplying priors by likelihoods gives the **posterior beliefs**, the update of the prior after seeing the evidence:

$$P(\text{cooking} \mid \textit{pepper}) \propto 0.01 \times 0.001 = 10^{-5} \quad P(\text{politics} \mid \textit{pepper}) \propto 0.12 \times 0.00002 = 2.4 \times 10^{-6}$$

→ One word was enough to overturn a twelve-to-one prior: the label is **cooking**.

# From One Word to a Whole Document

A document is not one word. A document  $d$  of  $T$  tokens is a sequence  $w_1, \dots, w_T$ , as when we counted n-grams, so what we need is

$$P(d \mid c) = P(w_1, w_2, \dots, w_T \mid c)$$

- As with n-grams, almost every document is **unseen**: we cannot count it.
- So we need to break the document down into pieces we *can* count.

# The Two Naive Bayes Assumptions

- **Bag-of-Words:** the position of a word does not matter, only which words occur and how often, as in the previous lecture
- **Conditional Independence:** given the class, words occur independently of each other

Together they turn the probability of the whole document into a product of one factor per token:

$$P(w_1, \dots, w_T \mid c) \approx \prod_{i=1}^T P(w_i \mid c)$$

- It is *naive* because the second assumption is false: *San* makes *Francisco* far more likely. It works surprisingly well anyway.
- In other words, every class gets its **own unigram language model**, and we ask which class's model makes the document most probable.

# Multinomial Naive Bayes

Putting it together, for a document of  $T$  tokens  $w_1, \dots, w_T$ :

$$\hat{y} = \operatorname{argmax}_{c \in Y} P(d \mid c) P(c) \approx \operatorname{argmax}_{c \in Y} \prod_{i=1}^T P(w_i \mid c) P(c)$$

Multiplying many small probabilities underflows, so we work in **log space**, as with probabilistic language models:

$$\hat{y} = \operatorname{argmax}_{c \in Y} \log P(c) + \sum_{i=1}^T \log P(w_i \mid c)$$

The score of each class is a **sum of one weight per token**, so it is a linear function of the word counts: Naive Bayes is a **linear classifier**.

# Training Naive Bayes

Training is **counting**. With  $N_c$  the number of training documents in class  $c$ :

$$\hat{P}(c) = \frac{N_c}{N} \qquad \hat{P}(w \mid c) = \frac{\text{count}(w, c) + 1}{\sum_{w' \in V} \text{count}(w', c) + |V|}$$

- $\text{count}(w, c)$ : all occurrences of  $w$  in all training documents of class  $c$
- $V$ : the vocabulary of the **whole** training set, all classes together

The  $+1$  is **Laplace smoothing**, seen with language models, and here it is not optional. It handles words that belong to  $V$  but have zero occurrences in a particular class.

- If *fantastic* never occurs in a negative training review,  $\hat{P}(\textit{fantastic} \mid \text{neg}) = 0$ , and every review containing it can never be negative, whatever else it says.
- Words that never occurred anywhere in the training set are not in  $V$  and are ignored at test time.

# Naive Bayes: Worked Example

|          | Class | Document                                     |
|----------|-------|----------------------------------------------|
| Training | —     | <i>just plain boring</i>                     |
|          | —     | <i>entirely predictable and lacks energy</i> |
|          | —     | <i>no surprises and very few laughs</i>      |
|          | +     | <i>very powerful</i>                         |
|          | +     | <i>the most fun film of the summer</i>       |
| Test     | ?     | <i>predictable with no fun</i>               |

- Priors:  $\hat{P}(-) = 3/5$ ,  $\hat{P}(+) = 2/5$
- $|V| = 20$ ; the negative documents contain 14 tokens, the positive ones 9
- "with" never occurs in training, so it is dropped: we score "*predictable no fun*"

# Naive Bayes: Worked Example

|                     | <i>predictable</i>                 | <i>no</i>        | <i>fun</i>                      |
|---------------------|------------------------------------|------------------|---------------------------------|
| $\hat{P}(w \mid -)$ | $\frac{1+1}{14+20} = \frac{2}{34}$ | $\frac{1+1}{34}$ | $\frac{0+1}{34} = \frac{1}{34}$ |
| $\hat{P}(w \mid +)$ | $\frac{0+1}{9+20} = \frac{1}{29}$  | $\frac{0+1}{29}$ | $\frac{1+1}{29} = \frac{2}{29}$ |

$$\hat{P}(-) \hat{P}(d \mid -) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5} \quad > \quad \hat{P}(+) \hat{P}(d \mid +) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.3 \times 10^{-5}$$

$$\hat{c} = \operatorname{argmax}_{c \in \{-, +\}} \hat{P}(c) \hat{P}(d \mid c) = -$$

The model predicts **negative**. "*fun*" pointed the other way, but "*predictable*" and "*no*" each count as independent evidence for the negative class.

# Naive Bayes: Strengths and Weaknesses

## Strengths

- **fast**: training is one pass of counting, prediction one sum per class
- works with **little training data**
- a **strong baseline** for text classification, especially for short texts and spam

## Weaknesses

- the independence assumption is false, so correlated words are counted as **separate evidence**: the probabilities come out **overconfident**, even when the ranking of the classes is right
- it models how each class generates documents, assuming features are independent given the class: overlapping features re-count the same evidence.
  - a word and the bigram containing it,
  - a word and its lemma,
  - a word and a lexicon count that includes it

# Supervised Learning in Text Analytics

---

## Linear Regression

# Simple Univariate Regression

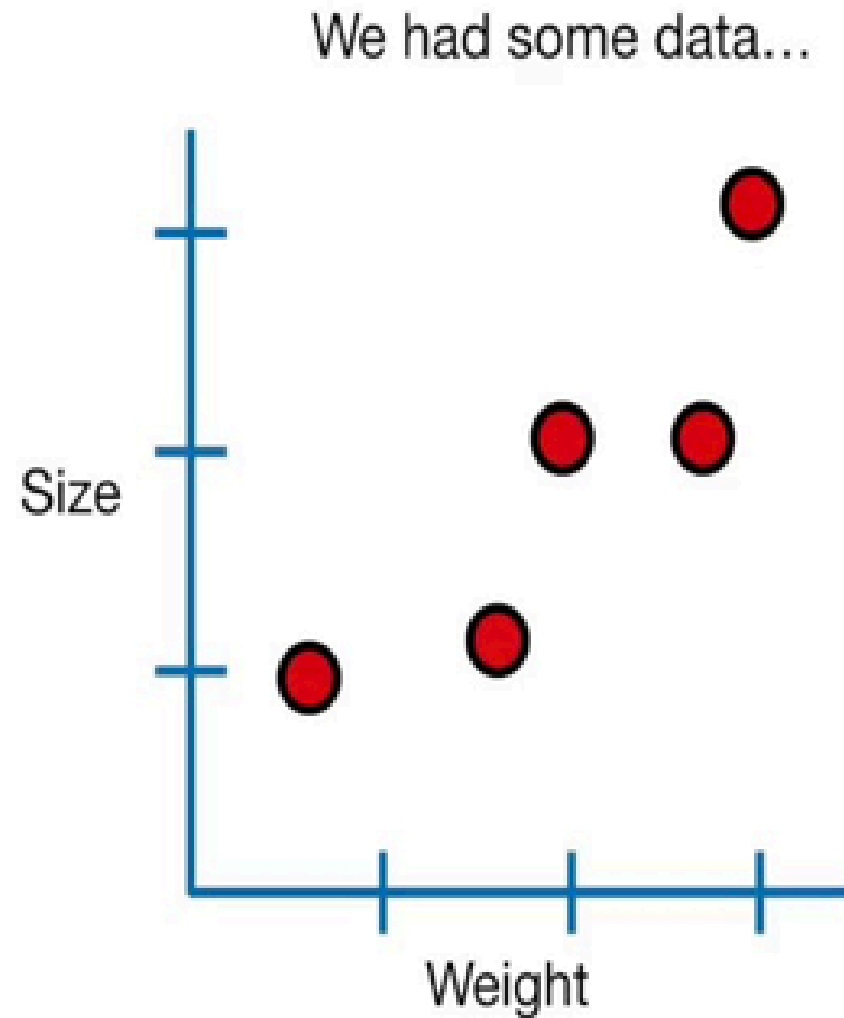
Let's start simple and suppose our dataset is composed of only:

- one feature column,  $\mathbf{x}$  (independent variable)
- one (continuous) target column,  $\mathbf{y}$  (dependent variable)

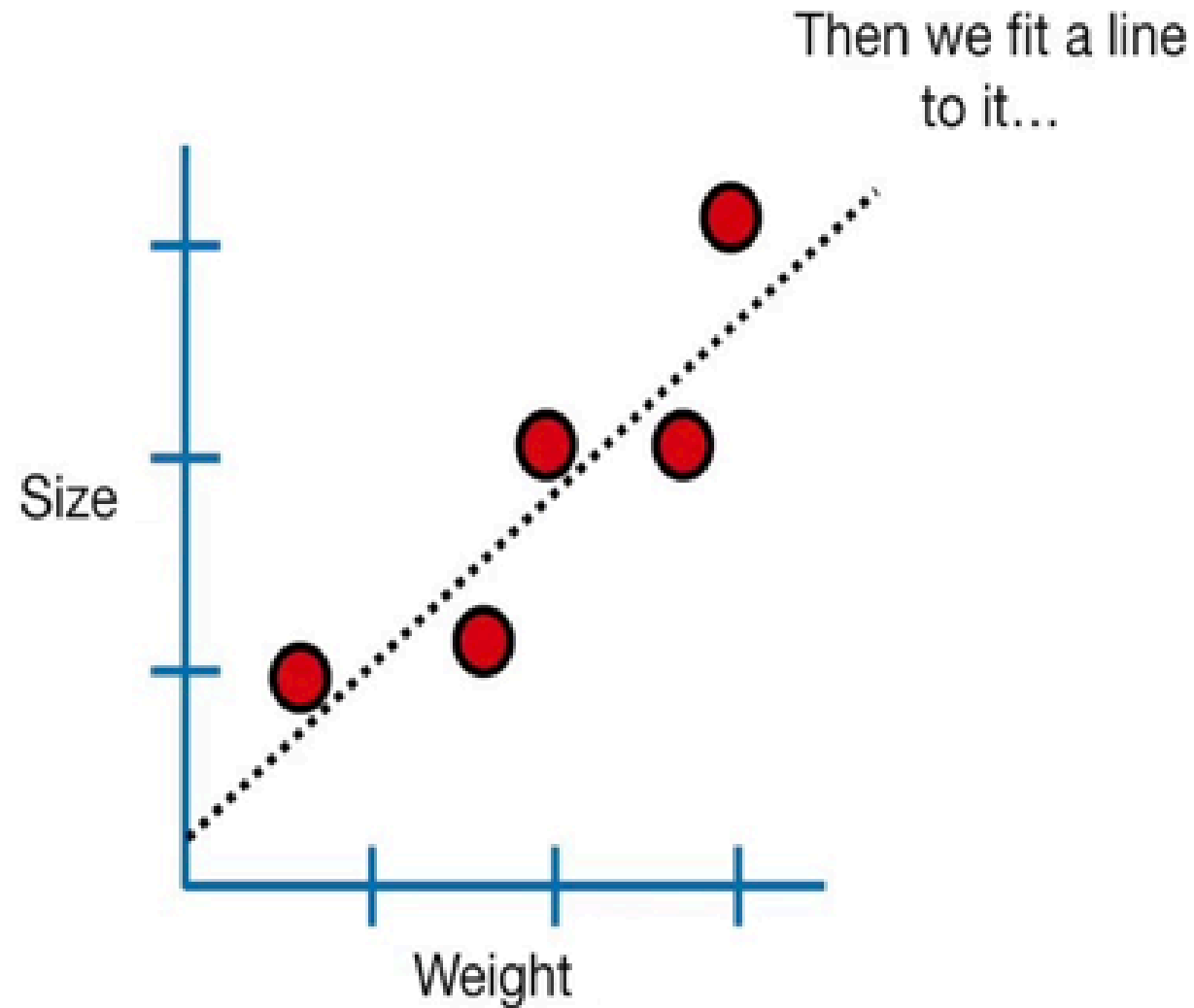
| $\mathbf{x}$ | $\mathbf{y}$ |
|--------------|--------------|
| $x_1$        | $y_1$        |
| $x_2$        | $y_2$        |
| $\dots$      | $\dots$      |
| $x_N$        | $y_N$        |

We want to predict each  $y$  given the corresponding  $x$ .

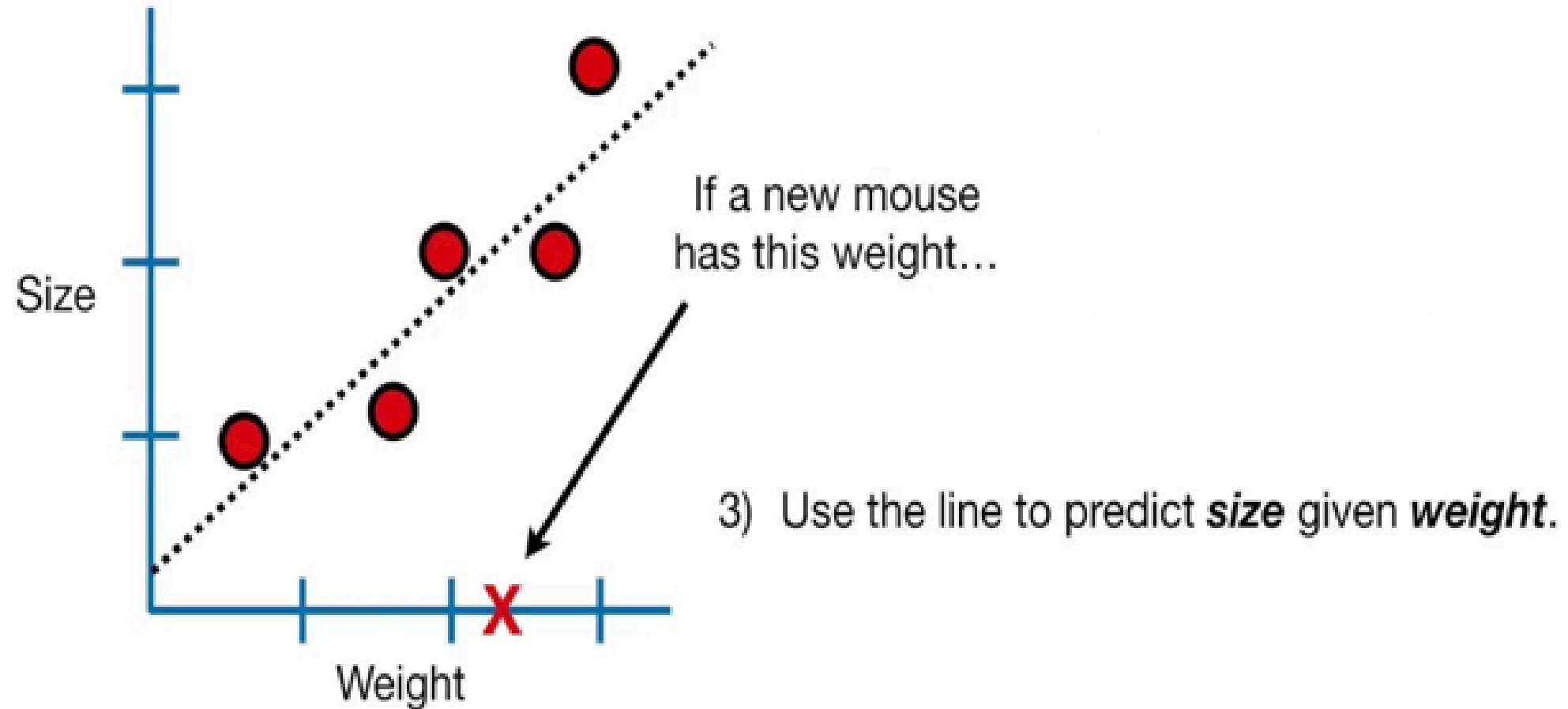
# What does it mean to linearly predict $y$ ?



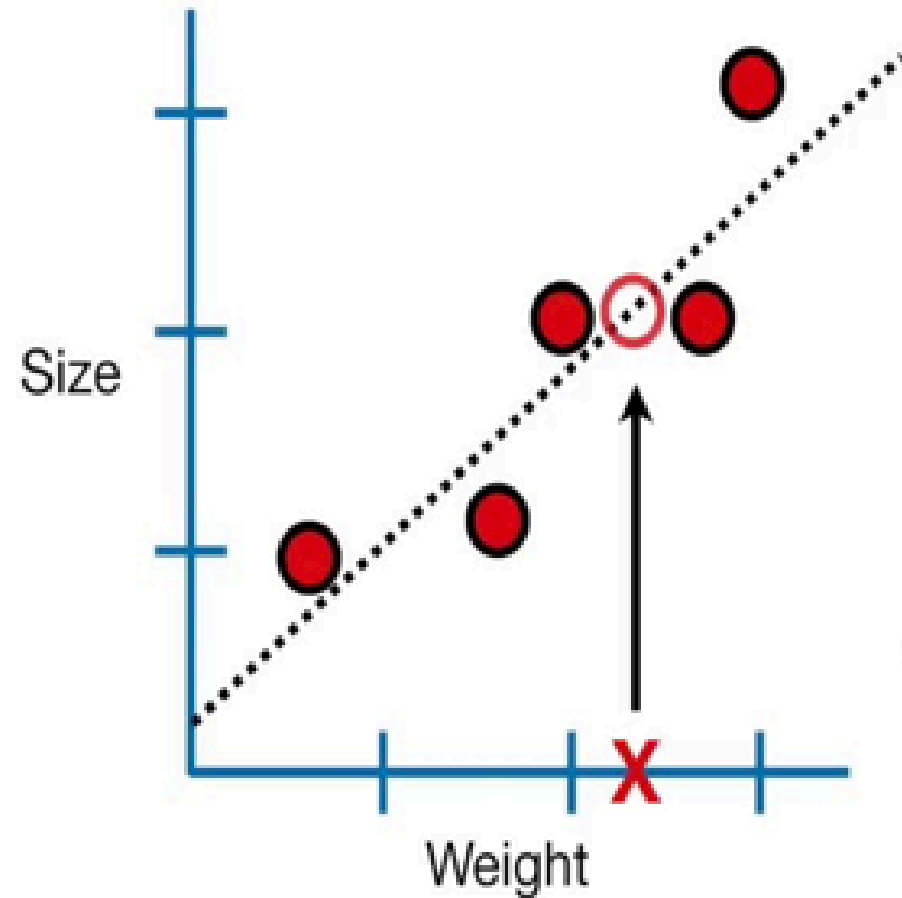
# What does it mean to linearly predict $y$ ?



# What does it mean to linearly predict $y$ ?

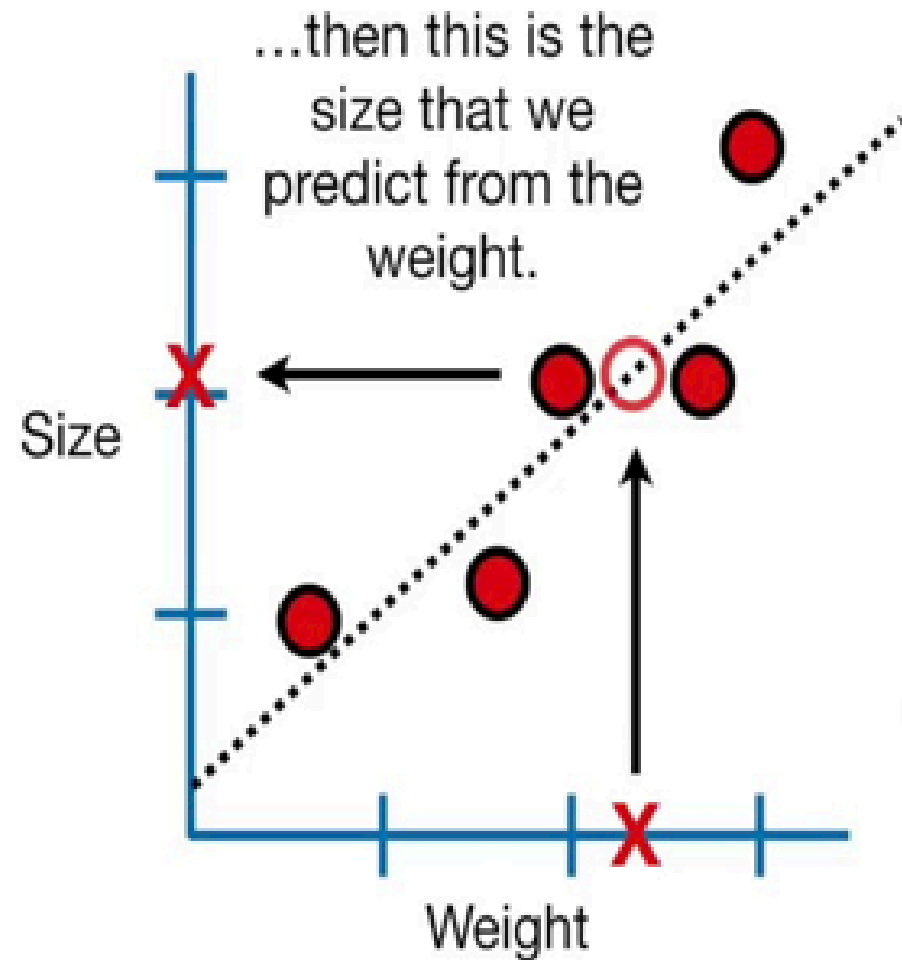


# What does it mean to linearly predict $y$ ?



3) Use the line to predict *size* given *weight*.

# What does it mean to linearly predict $y$ ?



3) Use the line to predict *size* given *weight*.

# The Line Equation

$$\underbrace{y}_{\text{dependent variable}} = \underbrace{m}_{\text{slope}} \underbrace{x}_{\text{independent variable}} + \underbrace{b}_{\text{bias}}$$

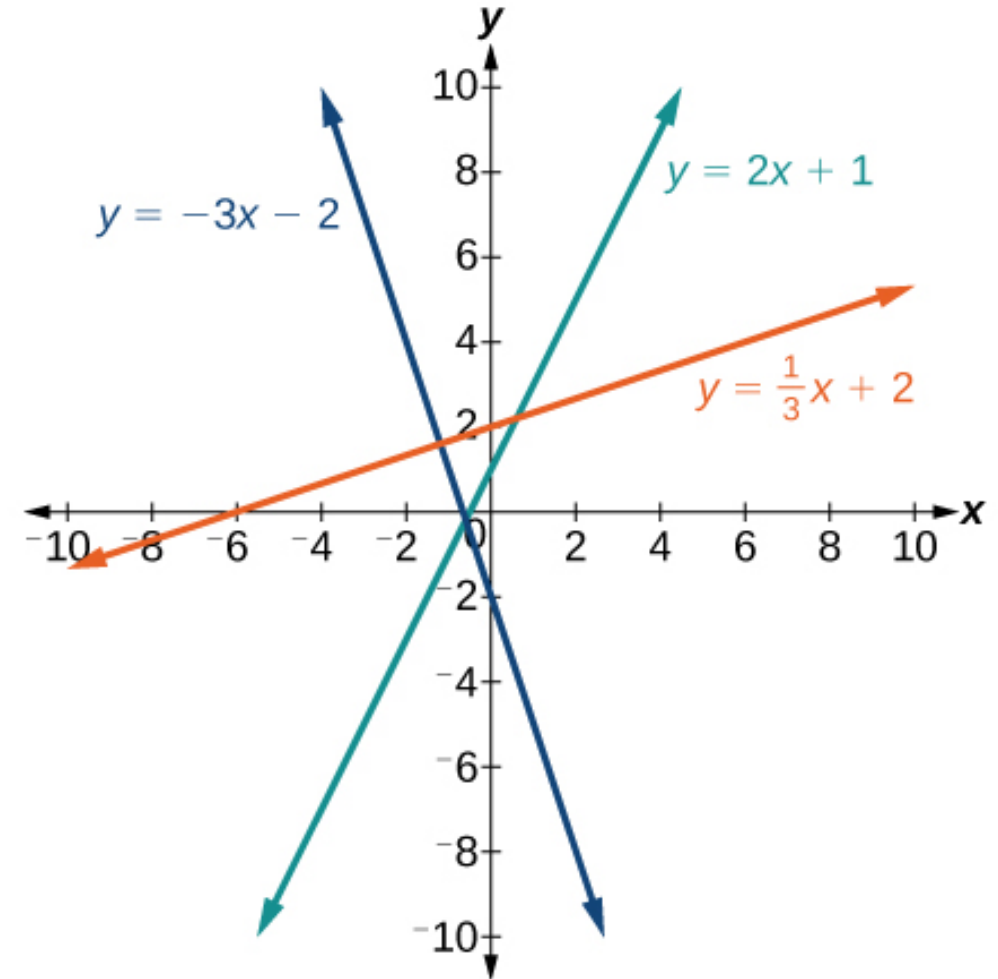
The line equation can be denoted in a few equivalent ways:

$$y = mx + b$$

$$y = ax + b$$

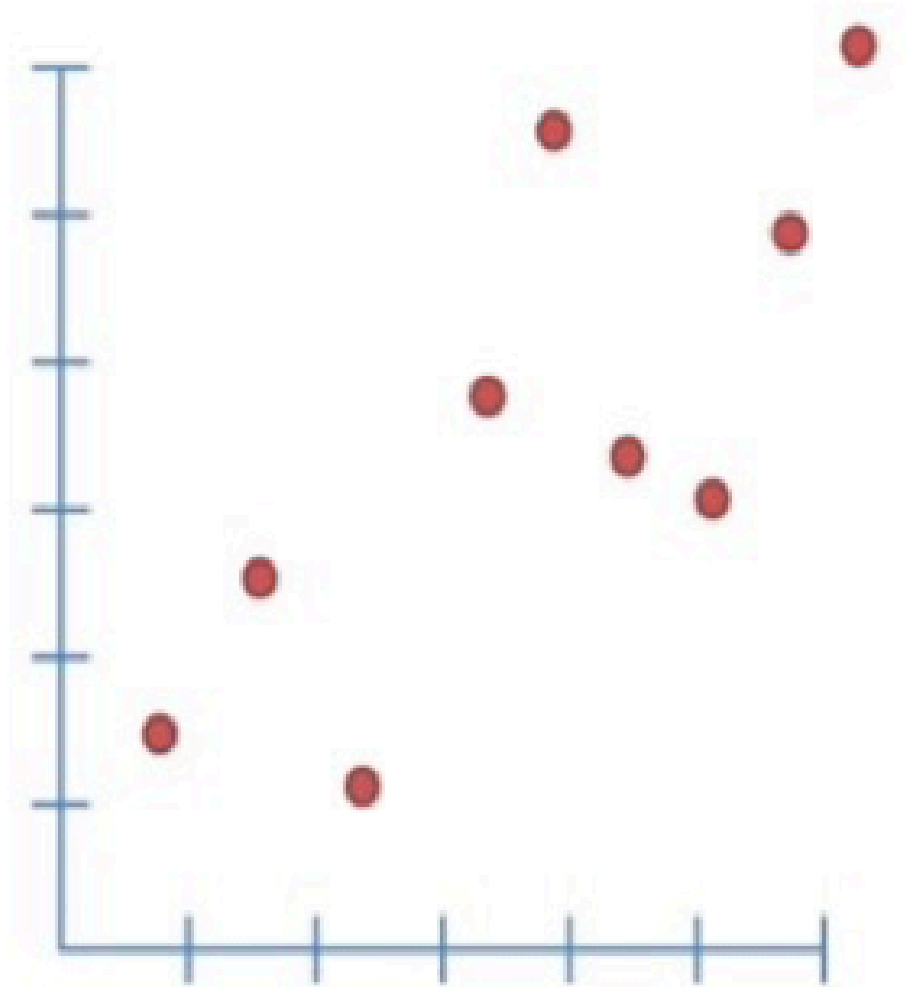
$$y = \beta_1 x + \beta_0$$

$$y = w_1 x + w_0$$

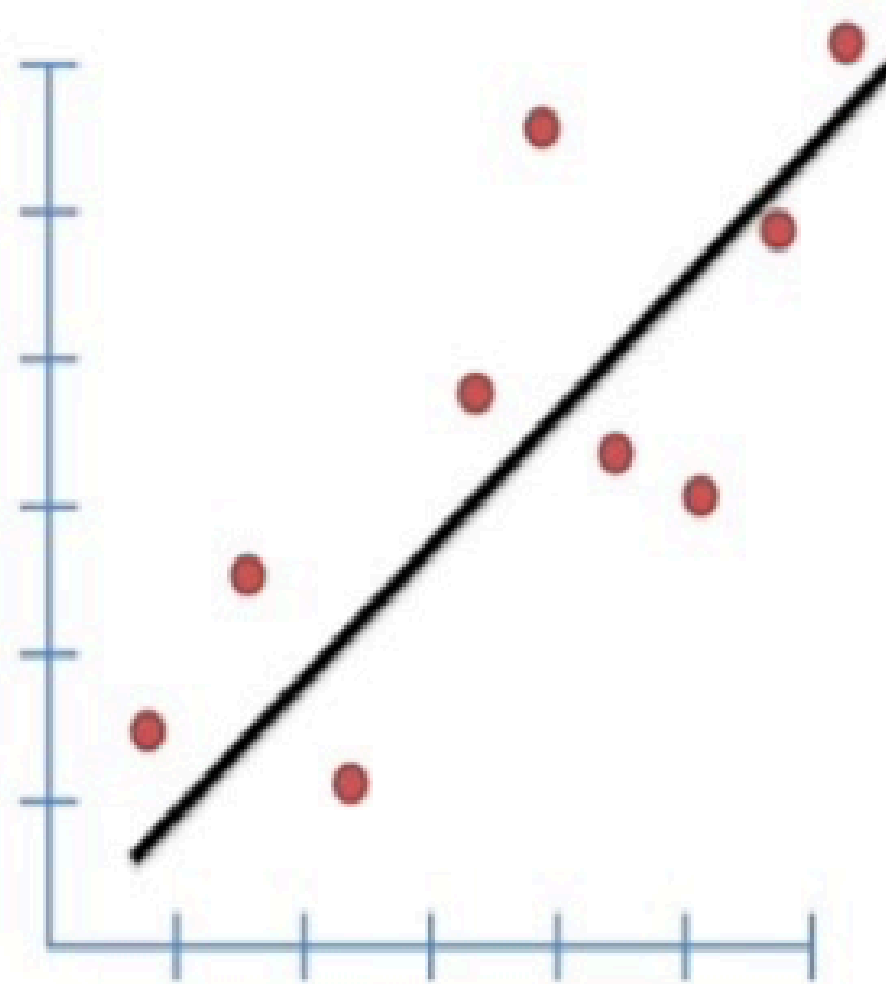


$w$  is a **weight** here, not a word.

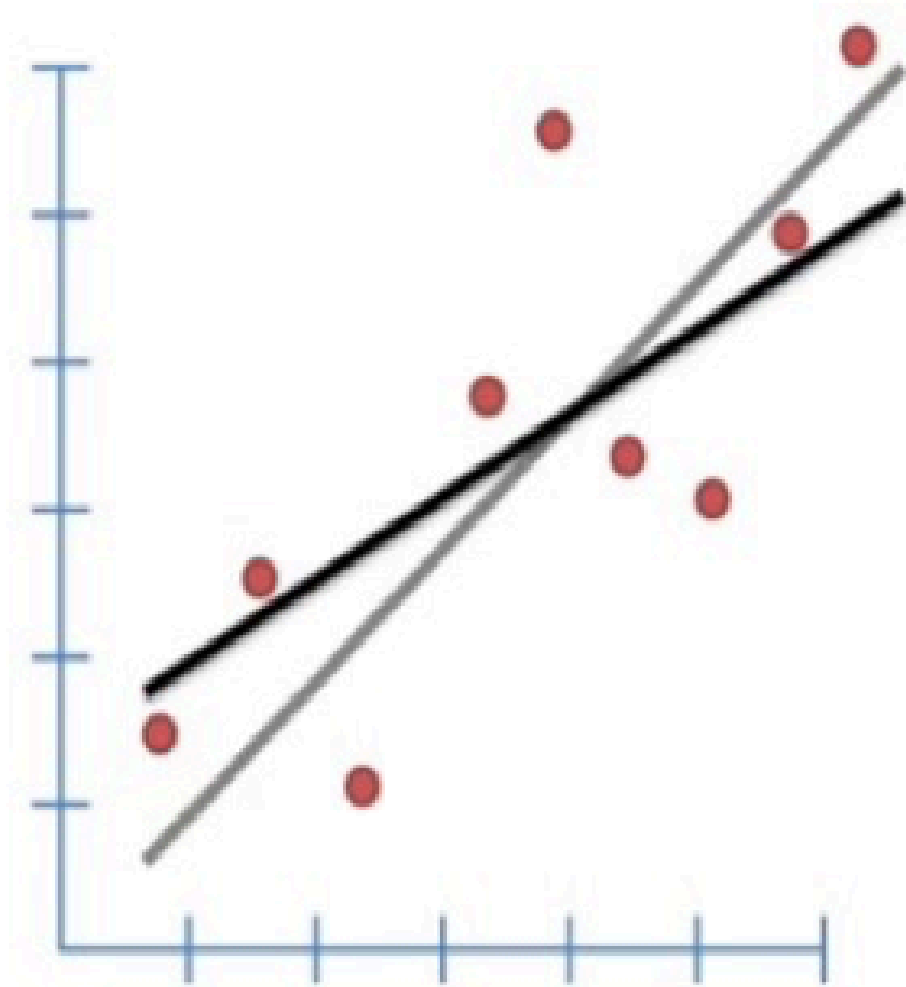
# Fitting a Linear Regression Model



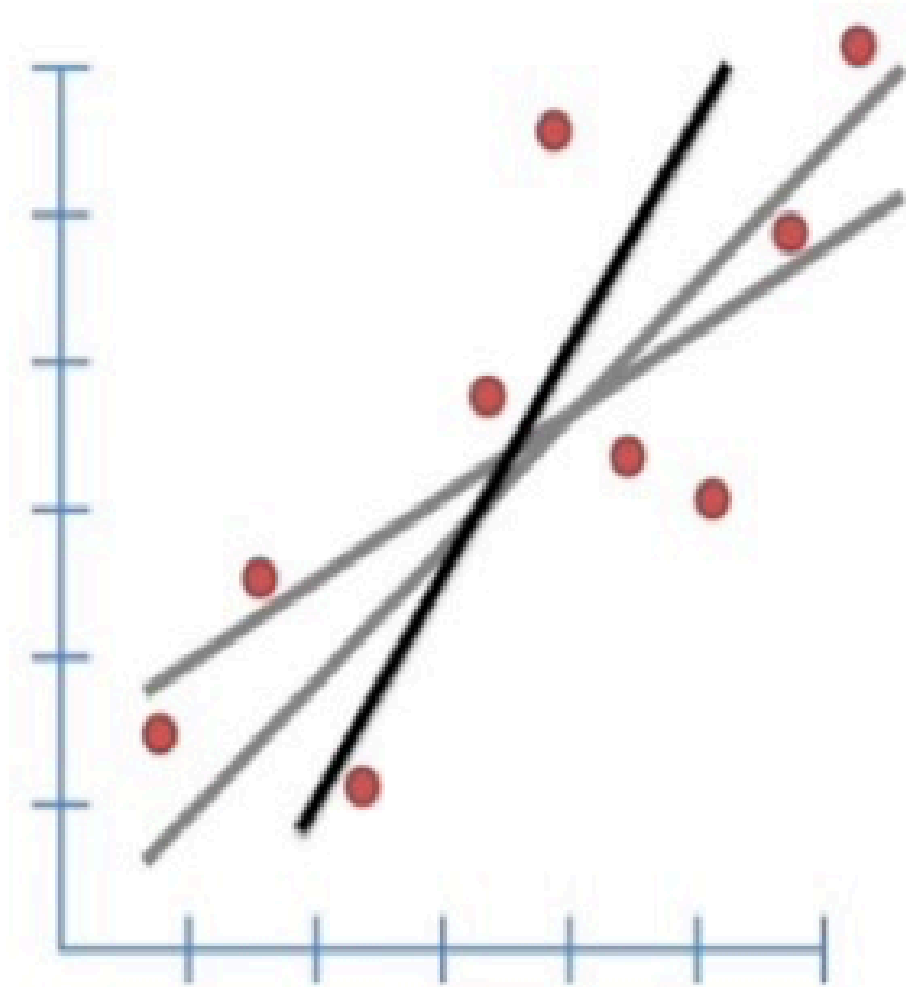
# Fitting a Linear Regression Model



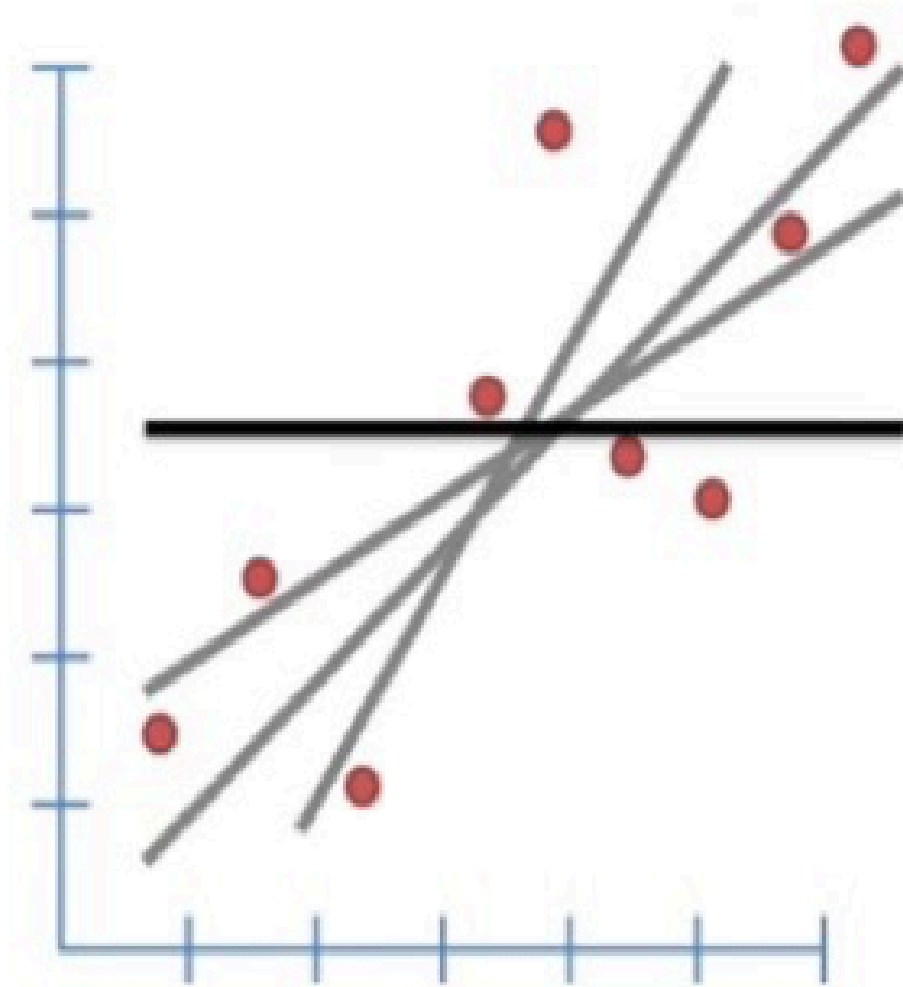
# Fitting a Linear Regression Model



# Fitting a Linear Regression Model



# Fitting a Linear Regression Model



# Least Squares

The best line is the one that minimizes the sum of squared errors.

Given a linear model  $f$ , we compute the sum of squared errors as:

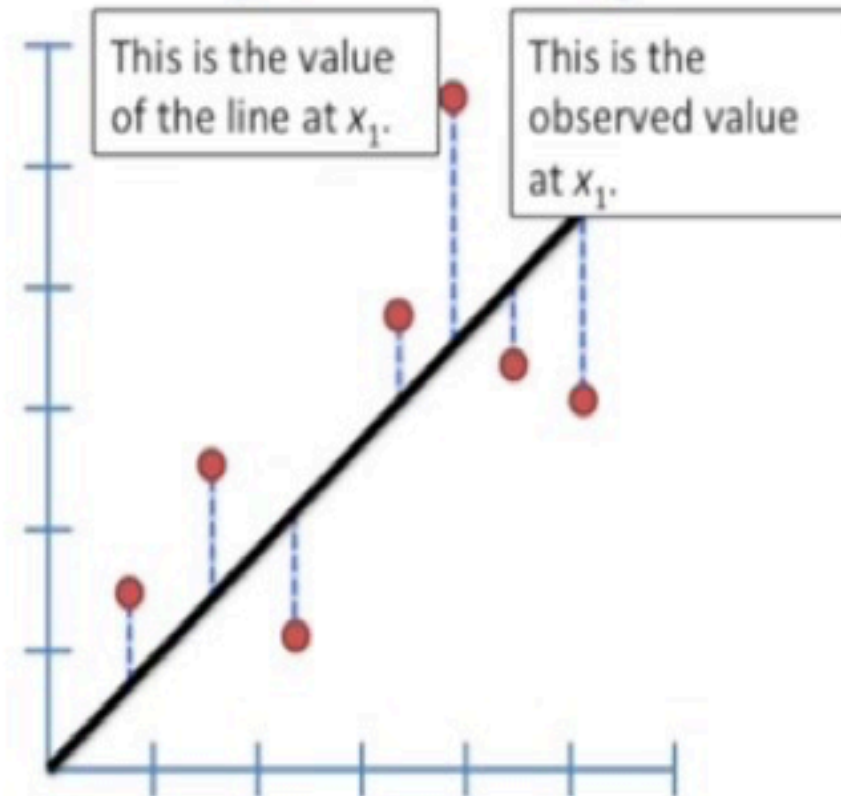
$$\text{SSE} = \sum_{i=1}^N (y_i - f(x_i))^2$$

Squared,

- so that errors above and below the line cannot cancel out,
- and so that large misses weigh more than small ones.

# Least Squares

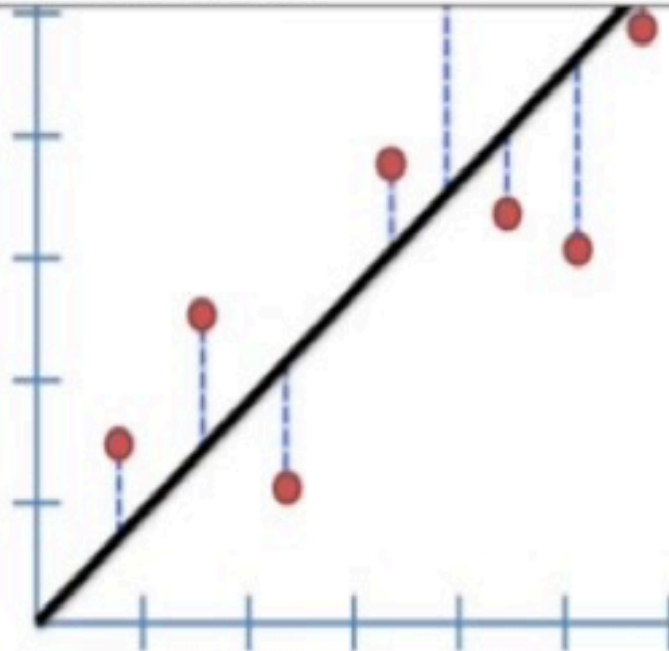
Sum of squared residuals =  $((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots$



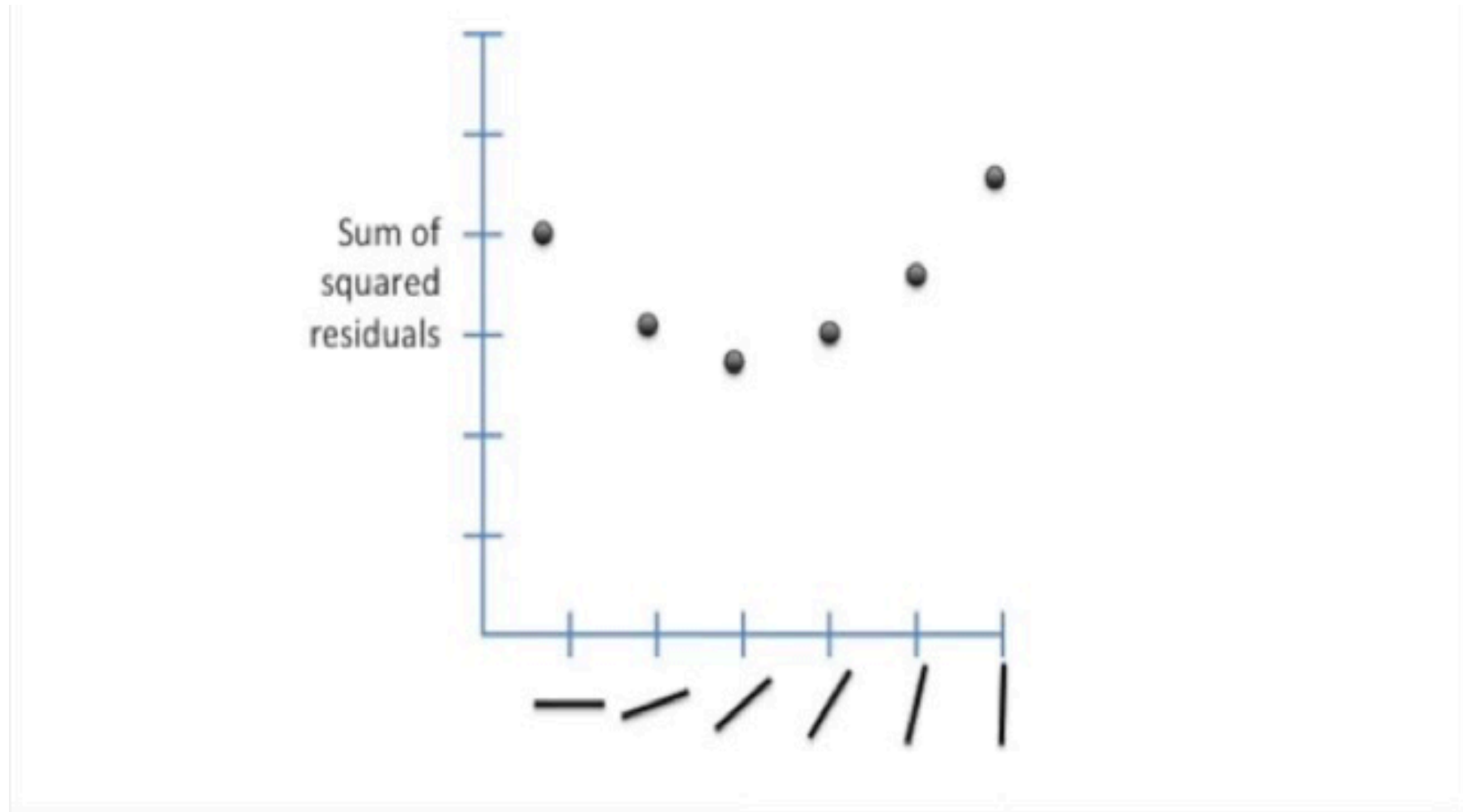
# Least Squares

Sum of squared residuals =  $((a \cdot x_1 + b) - y_1)^2 + ((a \cdot x_2 + b) - y_2)^2 + \dots$

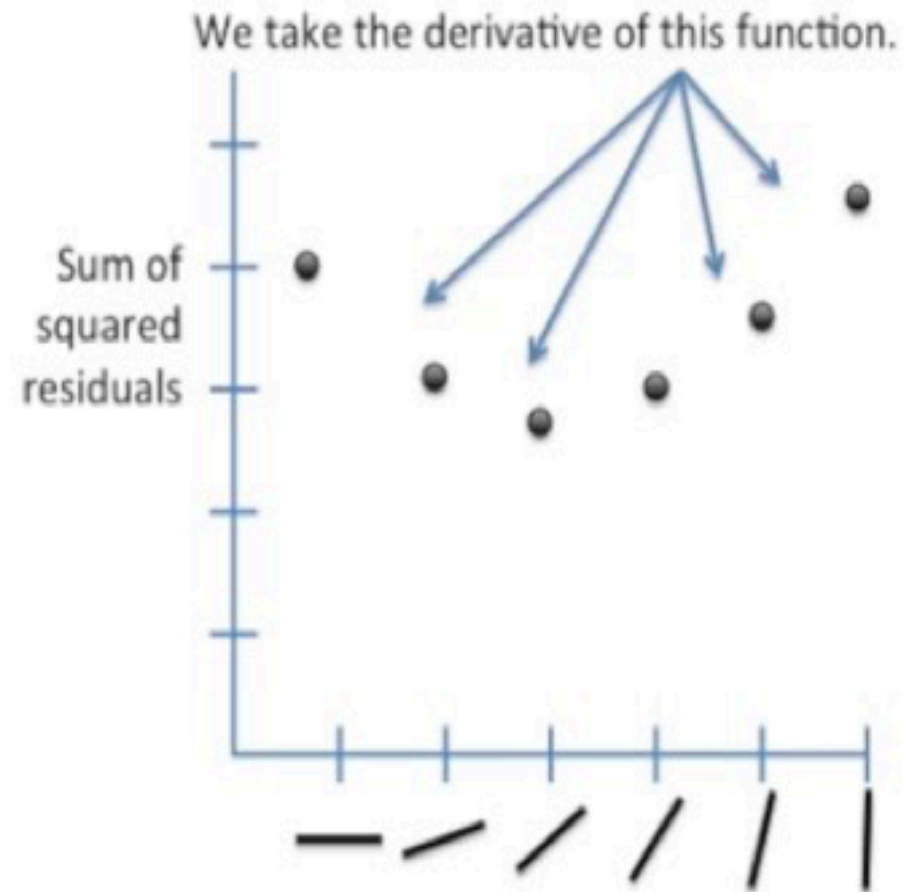
Since we want the line that will give us the smallest sum of squares, this method for finding the best values for " $a$ " and " $b$ " is called "Least Squares".



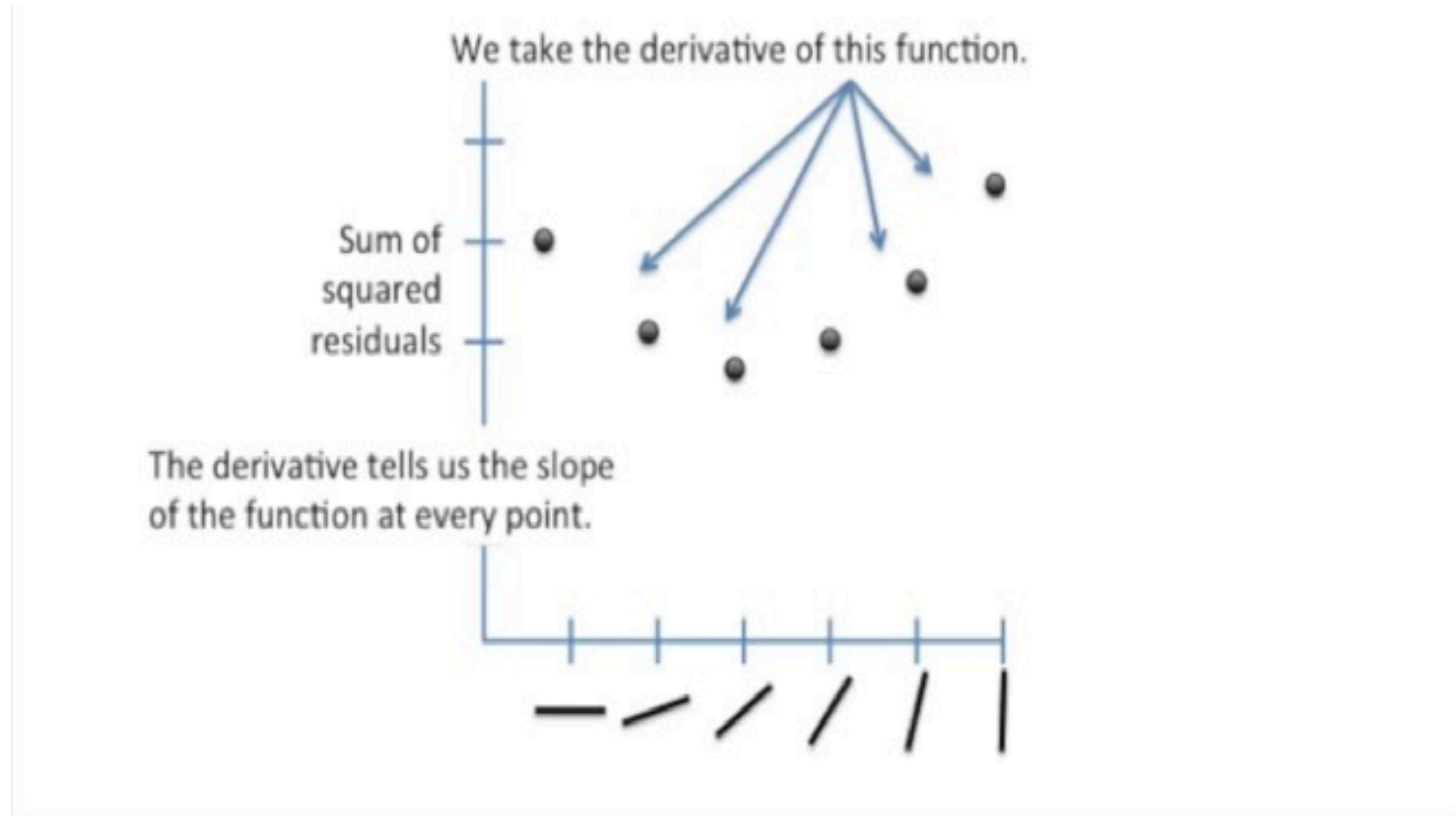
# Least Squares



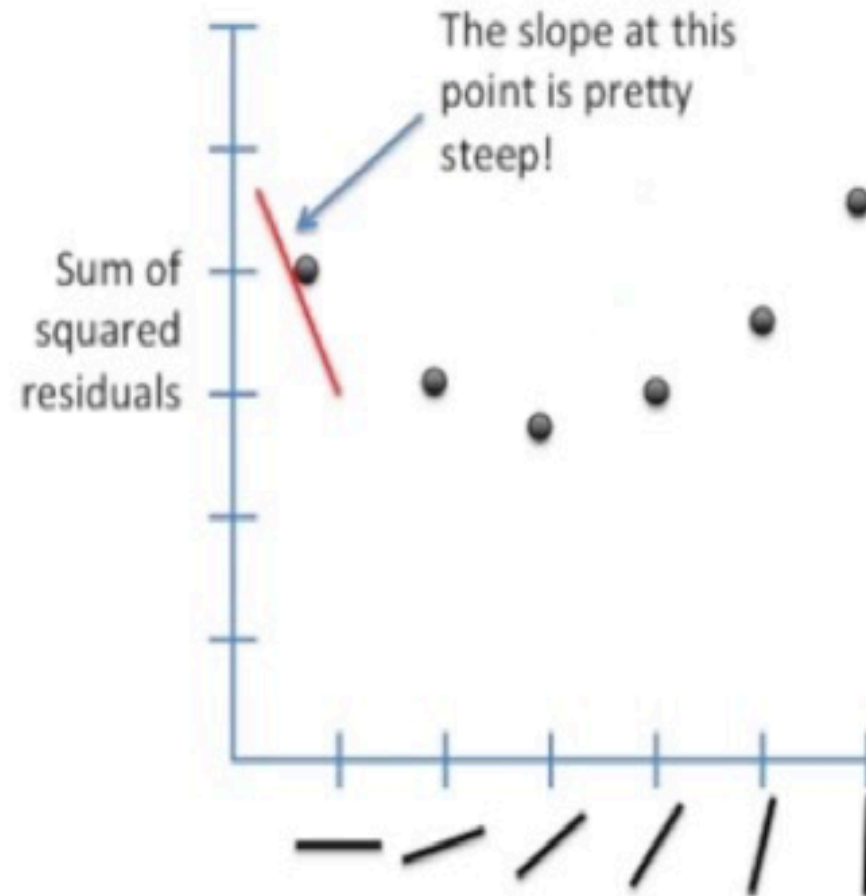
# Least Squares



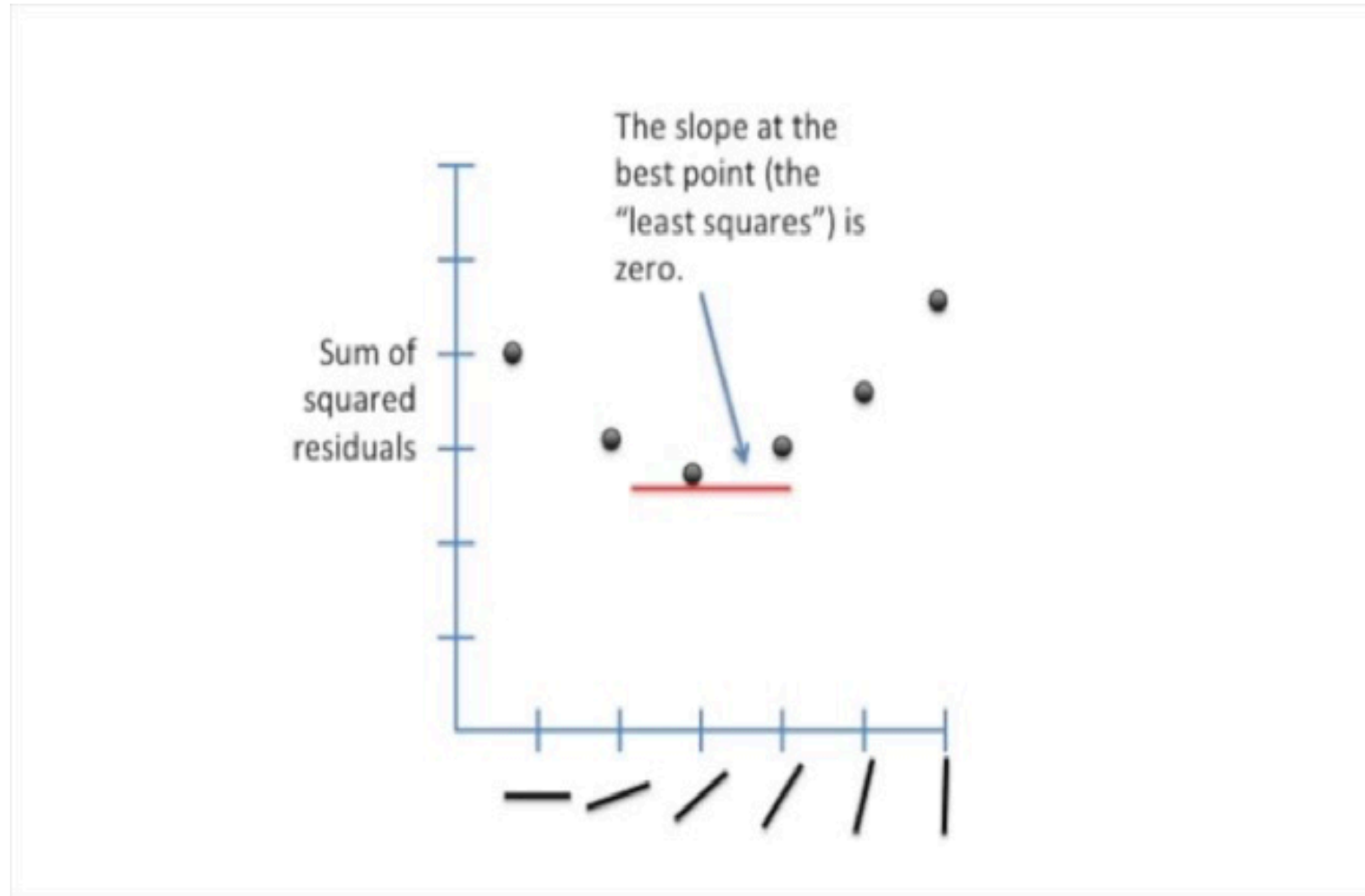
# Least Squares



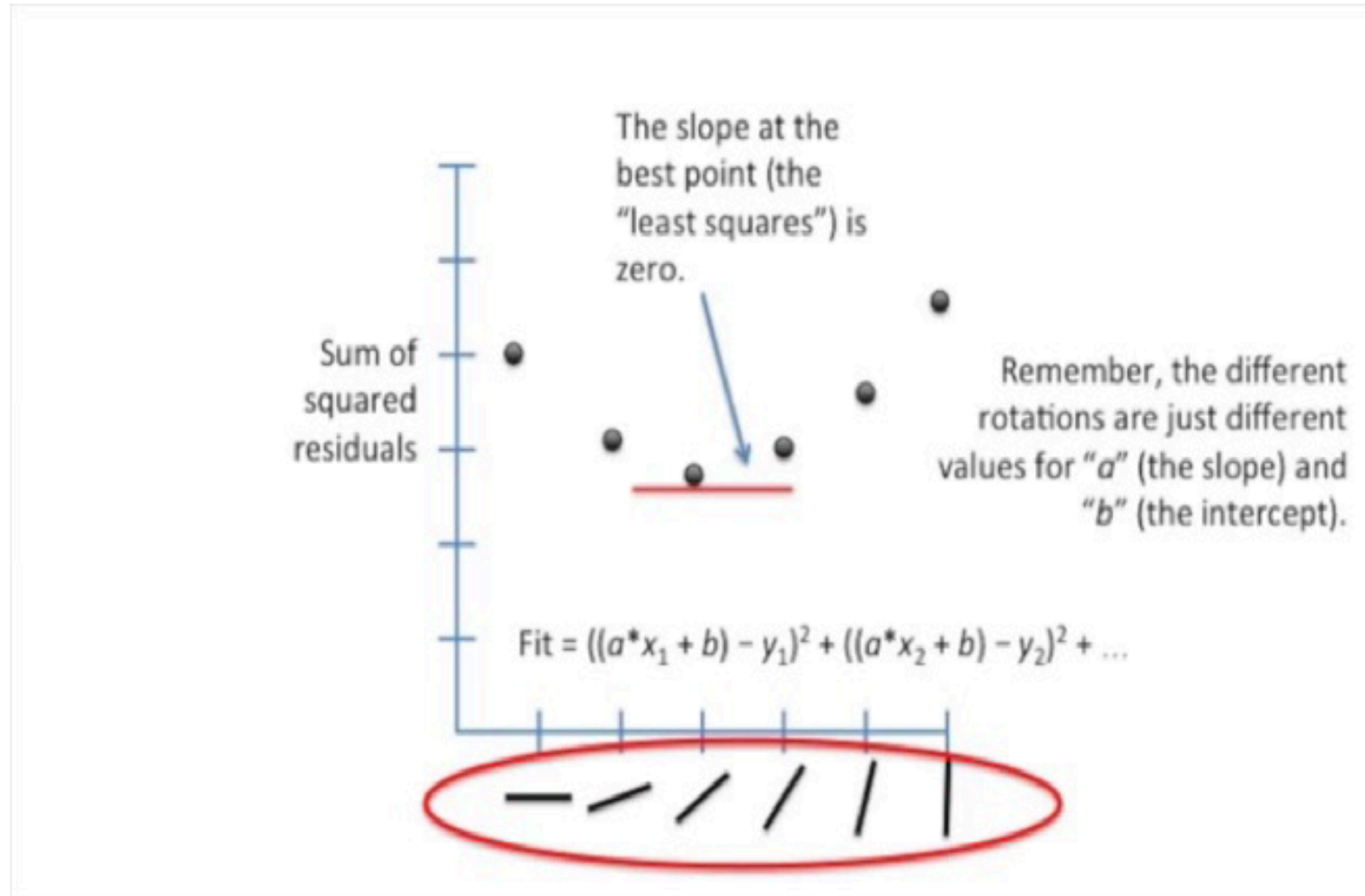
# Least Squares



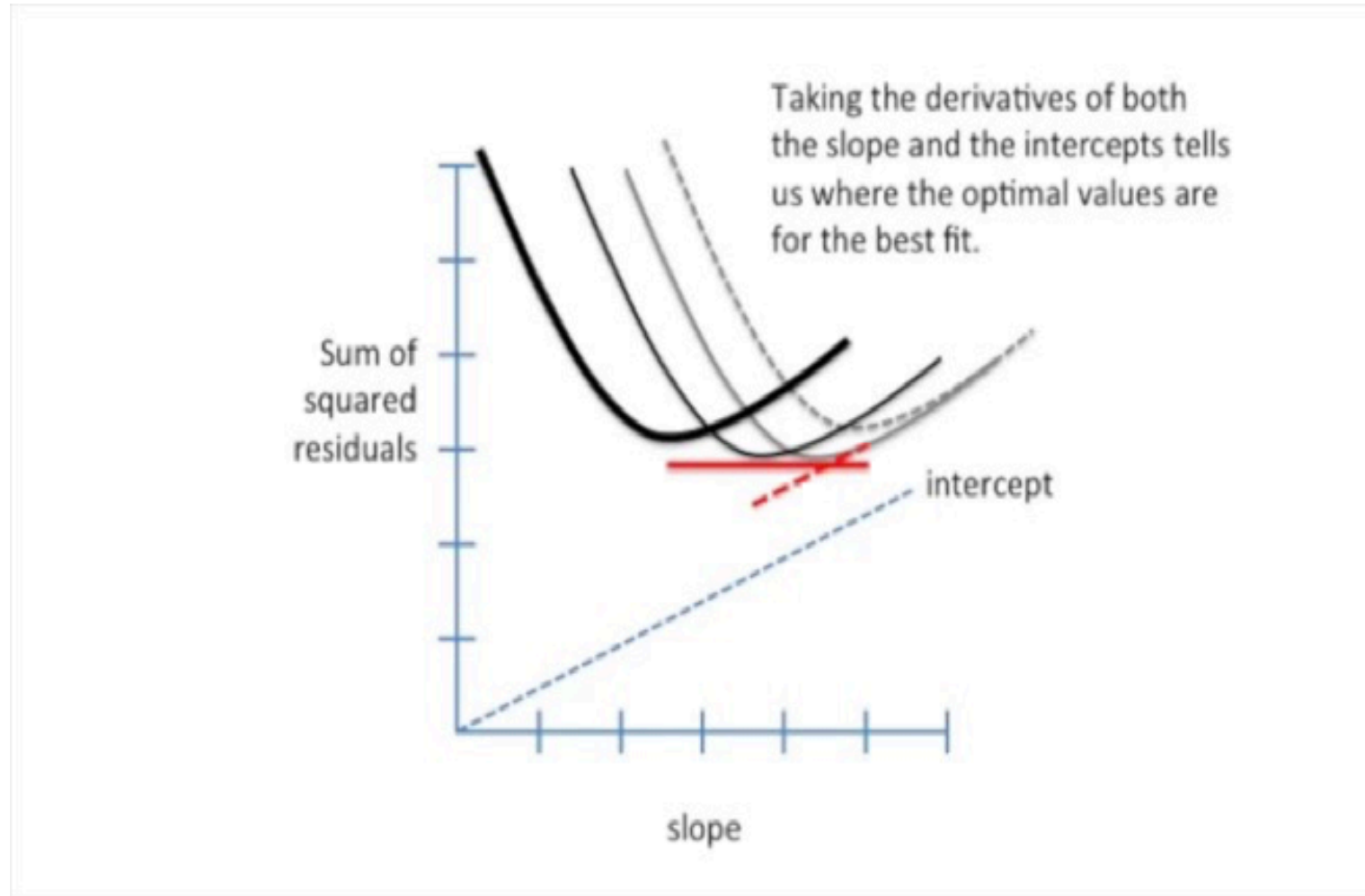
# Least Squares



# Least Squares



# Least Squares



# Solving Least Squares

We want to find optimal  $m, b$  such that the SSE is minimized:

$$(m^*, b^*) = \arg \min_{m, b} \sum_{i=1}^N (y_i - f(x_i))^2$$

This has a closed form solution obtained by setting the partial derivatives of SSE w.r.t.  $m$  and  $b$  to zero and solving:

$$m = \frac{N \sum(xy) - \sum x \sum y}{N \sum(x^2) - (\sum x)^2}, \quad b = \frac{\sum y - m \sum x}{N}$$

The solution is unique as long as the variance of  $x$  is not zero.

# Multiple Regression (Two Variables)

Now suppose our dataset is composed of:

- two feature columns:  $x_1, x_2$  (independent variables)
- one target column:  $y$  (dependent variable)

| $\mathbf{x}_1$ | $\mathbf{x}_2$ | $y$     |
|----------------|----------------|---------|
| $x_{11}$       | $x_{12}$       | $y_1$   |
| $x_{21}$       | $x_{22}$       | $y_2$   |
| $\dots$        | $\dots$        | $\dots$ |
| $x_{N1}$       | $x_{N2}$       | $y_N$   |

We want to predict each  $y_i$  given the corresponding pair  $(x_{i1}, x_{i2})$ .

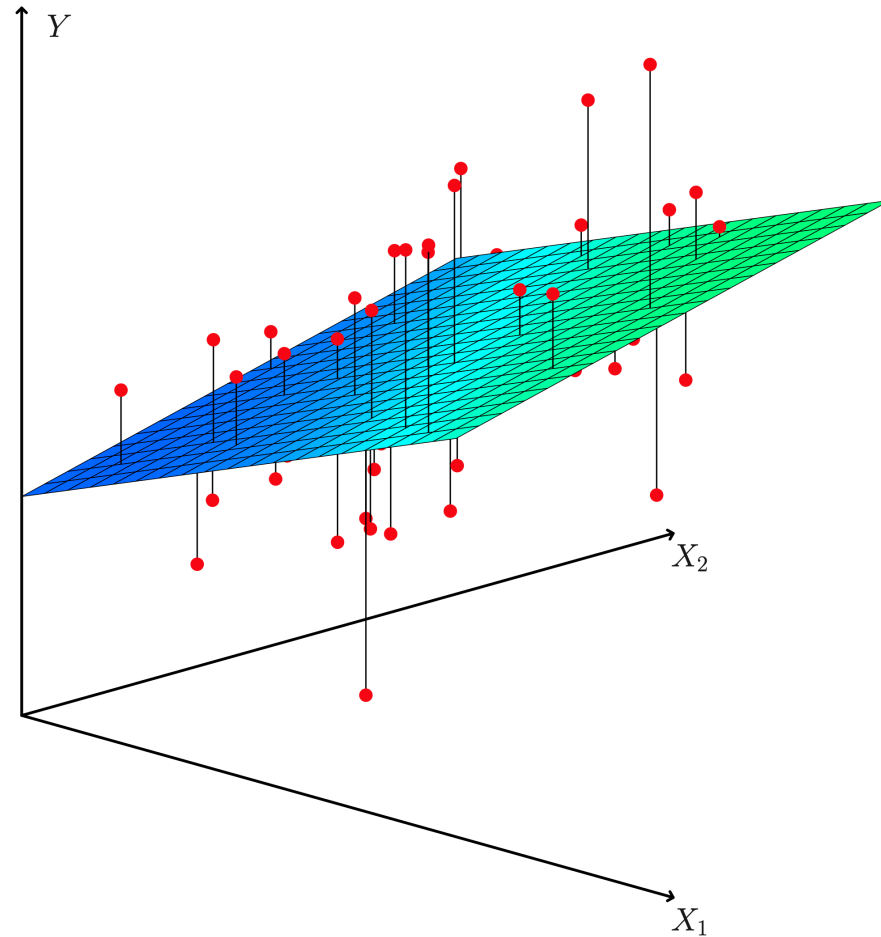
# Multiple Regression (Two Variables)

For each sample:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + b$$

where  $w_1, w_2$  are the weights and  $b$  is the bias.

We want to find the parameters  $w_1, w_2, b$  such that  $y_i \approx \hat{y}_i$ .



# Multiple Regression (General Case)

In the general case, with a dataset containing  $M$  features, each row is a vector

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{iM}) \in \mathbb{R}^M$$

We predict:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \dots + w_M x_{iM} + b = \sum_{j=1}^M w_j x_{ij} + b = \mathbf{w} \cdot \mathbf{x} + b$$

where  $w_j$  are the weights and  $b$  is the bias.

Least squares extends directly to multiple regression

# Interpreting Coefficients

For multiple regression,  $w_j$  indicates the change in  $y$  per unit change in  $x_j$ , holding all other variables constant.

Input scale matters, so feature-wise standardization allows a better comparison.

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

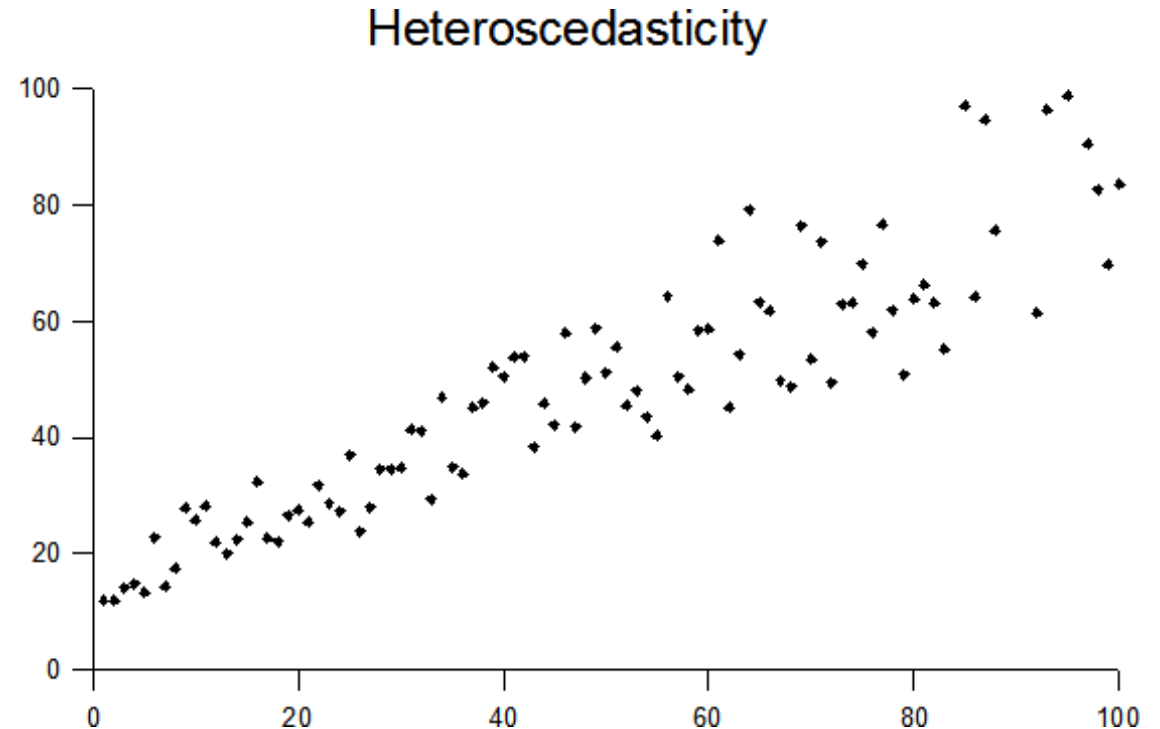
Benefits:

- Faster convergence if closed form solutions are not used
- Comparable coefficients
- Strongly recommended for Ridge/Lasso

# Assumptions of Linear Regression

A linear regression model assumes:

- **Linearity:** linear in parameters
- **Independent errors:** The error terms are statistically independent across observations
- **Homoscedasticity:** constant variance of errors
- **No strong multicollinearity:** predictors are not nearly exact linear combinations of each other
- **Normal residuals\*** (for statistical inference): the errors are normally distributed



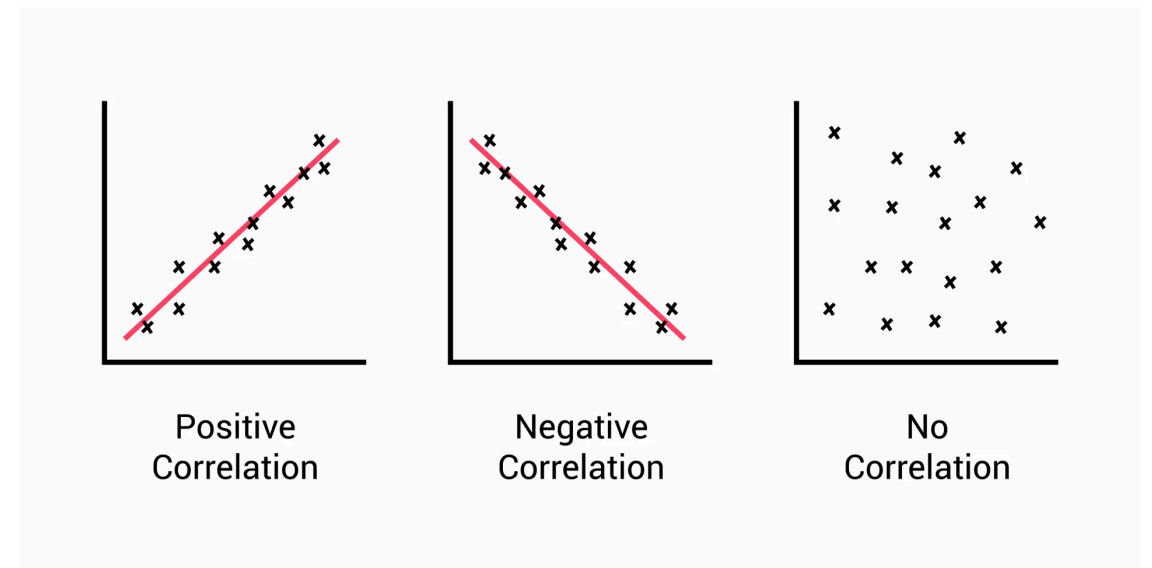
# Multicollinearity

When predictors are highly correlated:

- Coefficients become unstable
- Large variance in estimates
- Difficult interpretation

Mitigation:

- Ridge regression
- Feature selection
- Principal Component Analysis (PCA)



Some linear regression problems can benefit from **regularization**, i.e., discouraging large coefficient values.

- **Ridge** regression is a regularization method for ill-posed problems, particularly useful for mitigating multicollinearity.
- **Lasso** (Least Absolute Shrinkage and Selection Operator) performs both variable selection and regularization to enhance the prediction performance and interpretability of the resulting statistical model.

# Regularization

For multiple variables:

$$\text{Ridge: } \arg \min_{\mathbf{w}, b} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^M w_j^2$$

$$\text{Lasso: } \arg \min_{\mathbf{w}, b} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^M |w_j|$$

Regularization shrinks coefficients and reduces overfitting.

# Supervised Learning in Text Analytics

---

## Logistic Regression

# From Linear to Logistic Regression

Linear regression predicts a **number**, so it cannot answer a question like *is this review positive?*

- its output is unbounded, while a probability must lie in  $[0, 1]$
- its target is continuous, while a class is not

# Binary Classification in 2D

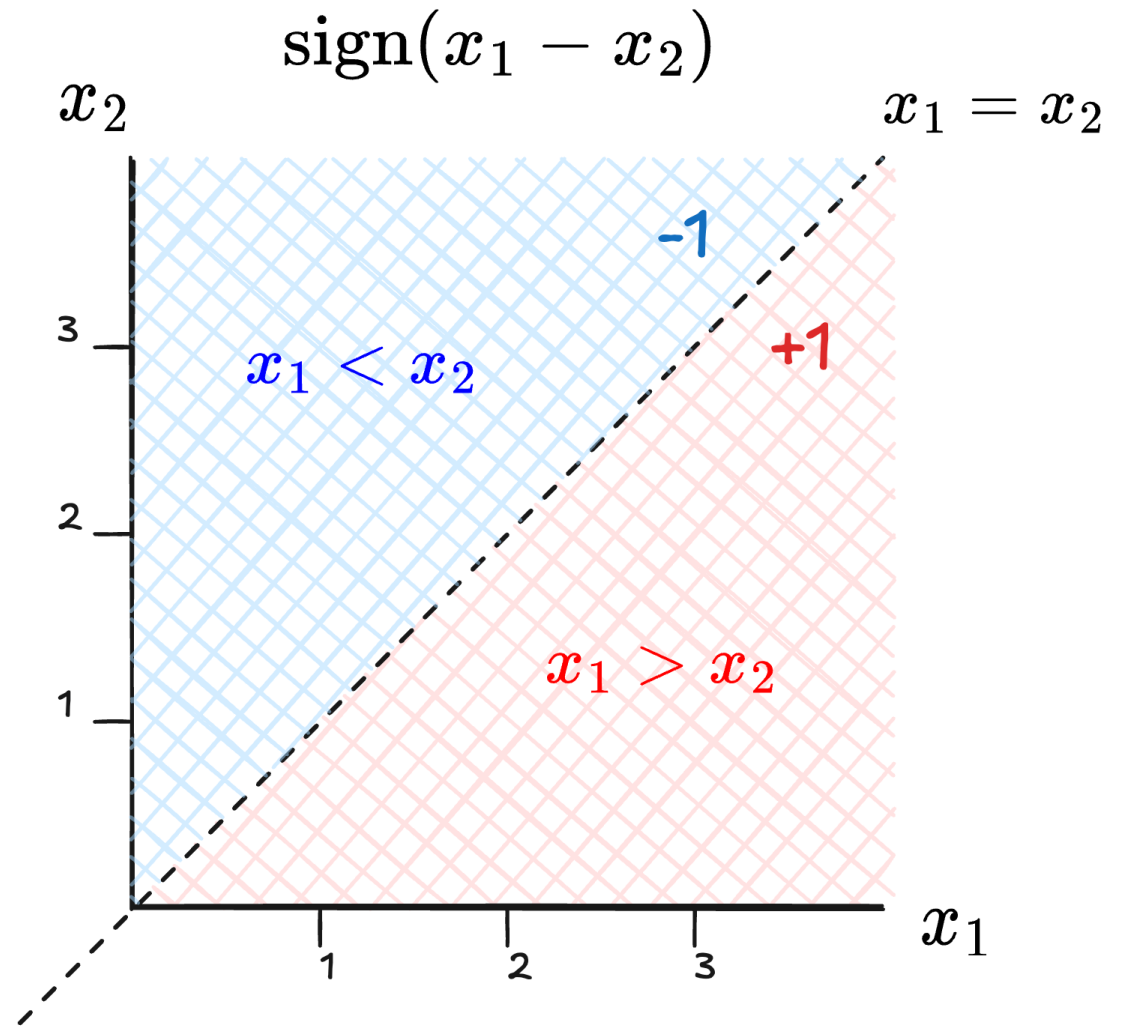
In  $\mathbb{R}^2$ ,  $w_1x_1 + w_2x_2 + b$  defines a line that separates space into two regions.

As a simple classification rule, we can use the sign function, which returns

- $+1$  if  $w_1x_1 + w_2x_2 + b \geq 0$ ;
- $-1$  otherwise:

$$\hat{y} = \text{sign}(w_1x_1 + w_2x_2 + b)$$

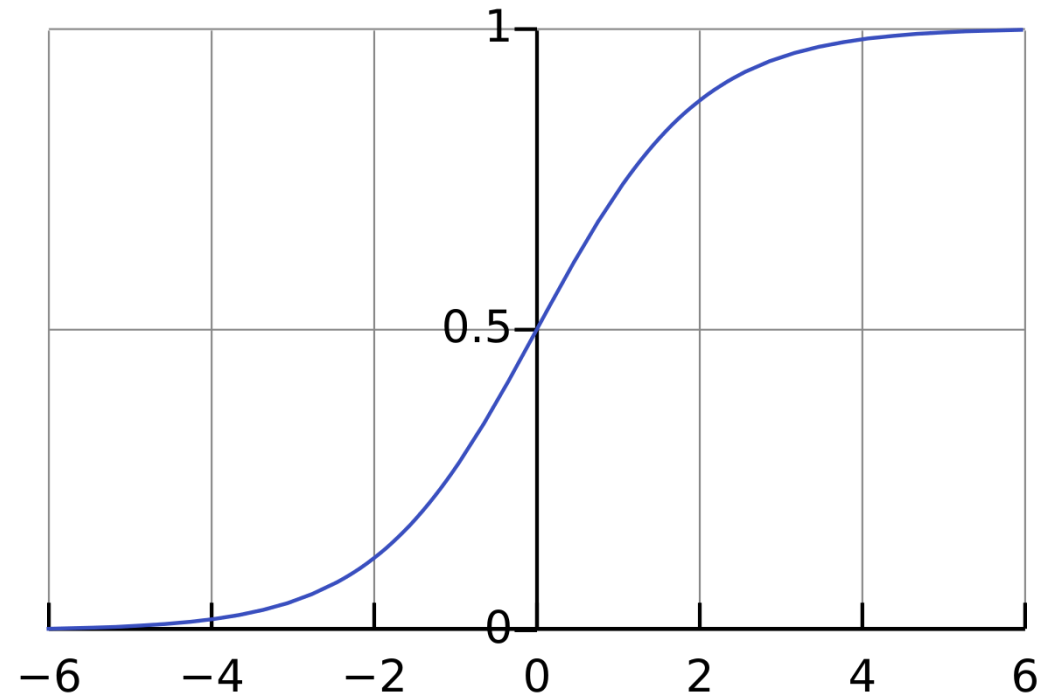
For example,  $\text{sign}(x_1 - x_2)$  is visualized on the right.



# Logistic Regression

- Instead of a step function, use a smooth function
- Logistic (sigmoid) function:
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$
- Interpreted as  $P(y = 1 \mid x)$
- A threshold is used for classification:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$



# Learning the Weights

Logistic regression models the **log-odds** as a linear function of the features:

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})} = \sum_{j=1}^M w_j x_j + b \iff P(y = 1 \mid \mathbf{x}) = \sigma \left( \sum_{j=1}^M w_j x_j + b \right)$$

The weights are chosen to **maximize the likelihood** of the training labels, equivalently to minimize the (binary) **cross-entropy** loss:

$$\ell(\hat{P}, y) = -[y \ln \hat{P} + (1 - y) \ln(1 - \hat{P})]$$

There is no closed-form solution: the weights are found by **gradient descent**, which we will meet again with neural networks.

# Logistic Regression: A Sentiment Example

*It's hokey. There are virtually no surprises, and the writing is second-rate. So why was it so enjoyable? For one thing, the cast is great. Another nice touch is the music. I was overcome with the urge to get off the couch and start dancing. It sucked me in, and it'll do the same to you.  $\rightarrow y = 1$  (Positive)*

|       | Feature                              | Value           | Weight |
|-------|--------------------------------------|-----------------|--------|
| $x_1$ | count of positive lexicon words      | 3               | 2.5    |
| $x_2$ | count of negative lexicon words      | 2               | -5.0   |
| $x_3$ | 1 if <i>no</i> is in the document    | 1               | -1.2   |
| $x_4$ | count of 1st and 2nd person pronouns | 3               | 0.5    |
| $x_5$ | 1 if <i>!</i> is in the document     | 0               | 2.0    |
| $x_6$ | $\ln(\text{number of tokens})$       | $\ln 66 = 4.19$ | 0.7    |

# Logistic Regression: A Sentiment Example

With the weights above and  $b = 0.1$ :

$$\begin{aligned}P(+ \mid \mathbf{x}) &= \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\&= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\&= \sigma(0.833) = 0.70\end{aligned}$$

$$\ell(0.70, 1) = -[1 \cdot \ln 0.70 + (1 - 1) \ln(1 - 0.70)] = -\ln 0.70 = 0.36$$

- The weights are **readable**: a negative lexicon word costs twice what a positive one earns ( $-5.0$  vs  $2.5$ ).
- The features **overlap freely**: lexicon counts, a single word, pronouns, punctuation, length. Naive Bayes, which must model how each feature is generated, could not combine them this easily.

---

Had the review been negative ( $y = 0$ ), the same prediction would cost  $\ell(0.70, 0) = -\ln(1 - 0.70) = 1.20$ : a confident wrong answer is penalized about 3× more than this correct one.

# Limitations

Binary Linear  
Classifiers can split  
the space only with a  
single straight line



To build a multiclass classifier for  $|Y| > 2$  classes, there are several strategies:

- **Softmax Regression** (Multinomial logistic regression): use a regression model that outputs a vector of scores, one per class. These scores are converted into class probabilities with the **softmax** function (see future lectures for more details!).
- **One-vs-Rest**: Train one binary classifier for each class vs all the others
- **One-vs-One**: Train one binary classifier for each class vs each other class

# Naive Bayes vs Logistic Regression

Both are probabilistic, both are linear in the features, but:

|                      | Naive Bayes                                                                               | Logistic regression                                           |
|----------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| Kind                 | <b>generative:</b> models $P(\mathbf{x} \mid c) P(c)$ , how each class produces documents | <b>discriminative:</b> models $P(c \mid \mathbf{x})$ directly |
| Training             | counting                                                                                  | optimization                                                  |
| Correlated features  | counted as independent evidence, so double-counted                                        | weight is shared among them                                   |
| Little training data | often <b>better</b>                                                                       | needs more                                                    |
| More training data   | levels off early                                                                          | usually <b>better</b>                                         |

Train Naive Bayes first as a **baseline**, then see if logistic regression beats it.

# Supervised Learning in Text Analytics

---

## Other Models

# What's the best decision boundary?

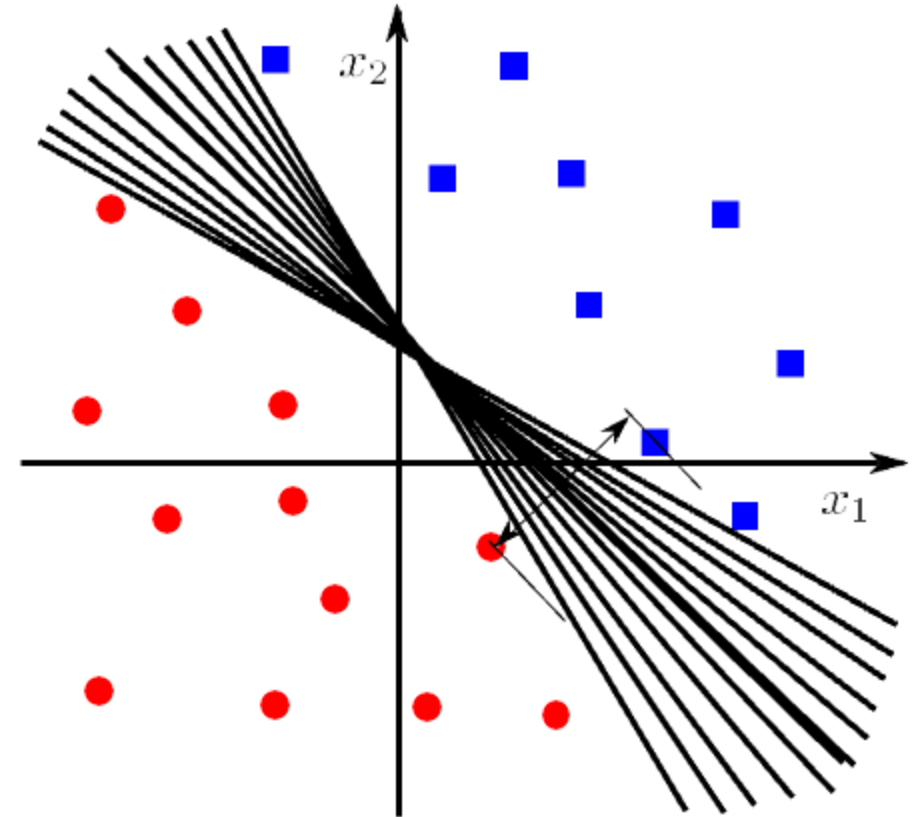
A linear classifier separates classes with a hyperplane (a line in 2D):

$$w_0 + \sum_{j=1}^M w_j x_j = 0$$

with the prediction rule being:

$$\text{sign} \left( w_0 + \sum_{j=1}^M w_j x_j \right)$$

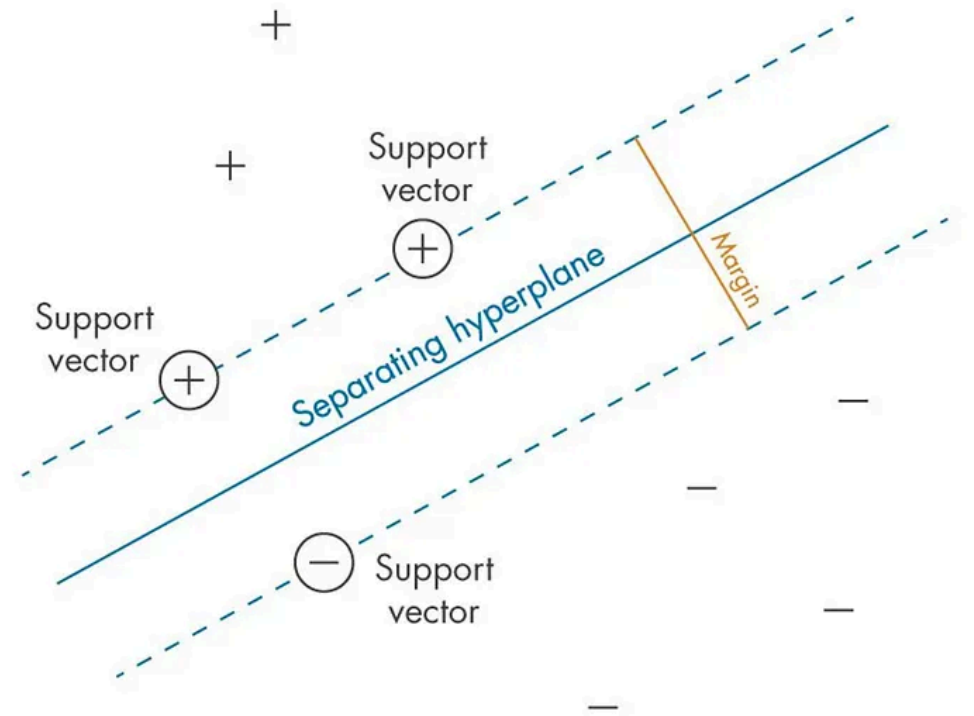
Given a linearly separable set of points, there are **infinite hyperplanes** that could solve the classification problem.



# Support Vector Machines

Which hyperplane should we choose?

- The **maximum margin** one: the widest band around the boundary that contains no training point.
- The points touching the band are the **support vectors**. They alone determine the solution.
- A **soft margin** lets a controlled number of points sit inside the band, or on the wrong side, so the method still works when the classes overlap.



Finding that hyperplane is a **constrained optimization** problem, solved with Lagrange multipliers.

# SVM: The Kernel Trick

What if no hyperplane separates the classes?

- Map the data to a **higher-dimensional** space,  $\mathbf{x} \mapsto \phi(\mathbf{x})$ , where a linear boundary is enough.
- The SVM never needs the new coordinates, only **dot products** between them. A **kernel** returns those directly:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i) \cdot \phi(\mathbf{x}_j)$$

so  $\phi$  is never computed.

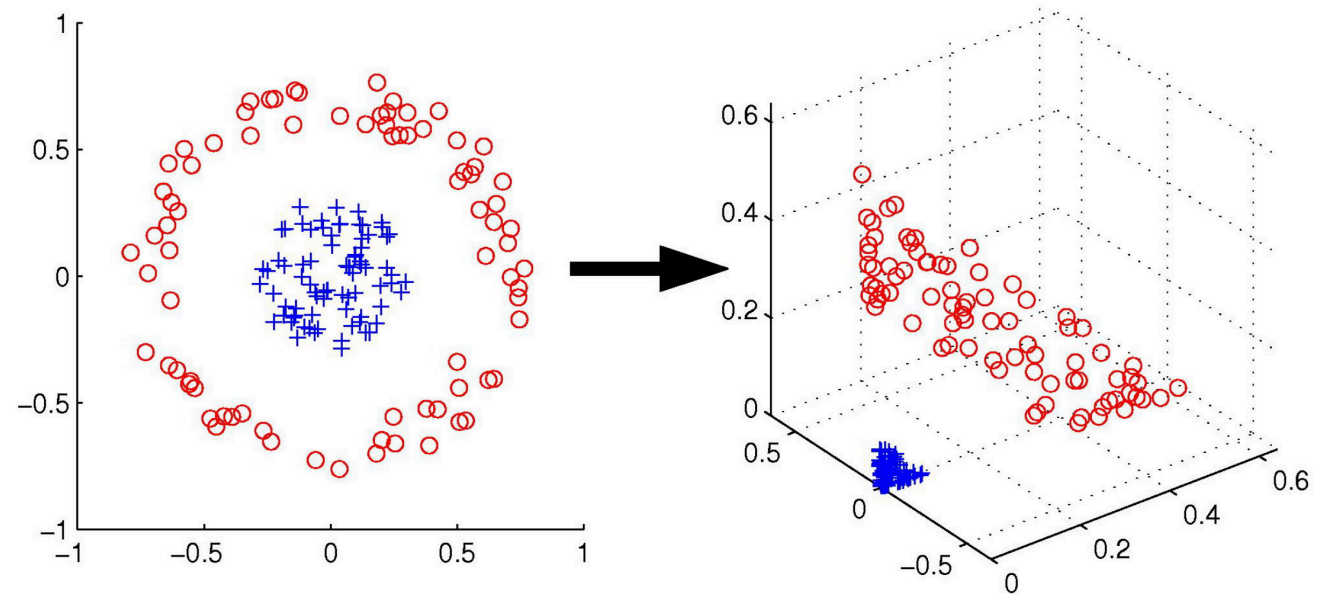


Figure 1: Transforming the data can make it linearly separable

# SVM: Kernels for Text

A kernel is a similarity between two objects: it does **not** need fixed-length vectors. It can be defined directly on text, with no bag of words in between.

- **Spectrum kernel:** how many character  $k$ -grams do two documents share? Robust to spelling and morphology, and it needs no tokenizer.
- **Subsequence kernel:** shared subsequences, gaps allowed, so *text analytics* also matches *text for analytics*, with a penalty for the gap.
- **Tree kernels:** how many subtrees do two parse trees share? This compares the **syntactic structure** we built when parsing, instead of the words.

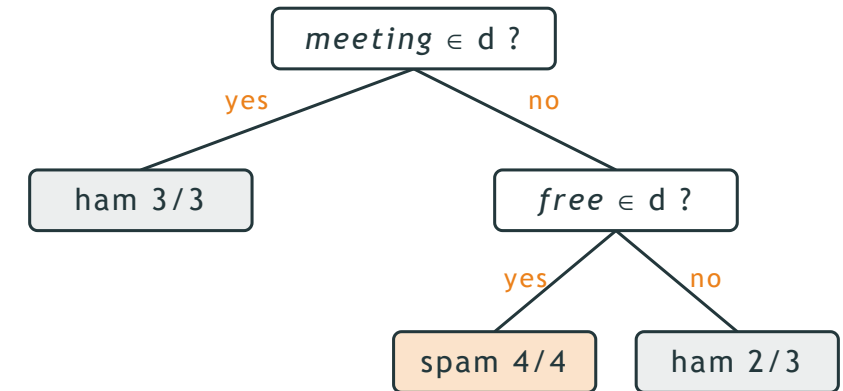
The only formal requirement is that the kernel matrix be **symmetric and positive semidefinite**, which keeps the optimization convex.

They are rarely used in practice: the kernel matrix is quadratic in the number of documents, and a **linear SVM on TF-IDF is hard to beat**.

# Decision Trees

A **decision tree** classifies by asking questions about the features, one at a time.

- each **internal node** tests one feature: does the email contain *meeting*?
- each **branch** is an answer, and each **leaf** predicts the majority class of the training documents that reach it



The tree is grown **greedily**: at every node, pick the feature whose split makes the children as **pure** as possible, then repeat on each child. Stop when a node is pure, too small, or too deep. The usual purity criterion is **information gain**, which we define in the feature selection section.

Every path reads as a rule: *IF*  $\text{meeting} \notin d$  *AND*  $\text{free} \in d$  *THEN* spam

# Why Decision Trees for Text?

**Because they explain themselves.** The model *is* its explanation: a few readable rules, written in the words of the documents. Trees also:

- mix feature types with no scaling: word counts, linguistic features, metadata such as the sender or the number of links
- are natively **multiclass**

**But on bag-of-words they are weak.**

- With tens of thousands of sparse features and one word per split, trees grow deep, overfit, and change completely when the training set changes a little.
- **Ensembles** (random forests, gradient boosting) recover the accuracy and give up the readability that was the point.

Rule of thumb: trees for **few, meaningful** features; linear models for the **full vocabulary**.

# Other Models

Once documents are **vectors**, a labeled corpus is a table of numbers, and **any classifier for tabular data applies**: k-nearest neighbours, random forests, gradient boosting, perceptrons, neural networks (later in the course).

What text changes is the **shape of the table**:

- **very high dimensional**: one column per word type
- **sparse**: almost every entry is zero
- **correlated**: words come in families

That shape is why the simple linear models are hard to beat here:

- Naive Bayes, logistic regression and linear SVMs all cope well with many sparse features,
- while methods that rely on distances in a dense, low-dimensional space (k-nearest neighbours) or on one feature at a time (a single tree) struggle.

# Sources: Textbooks

- Linear Regression: Applied Linear Regression, Weisberg, ch. 2
- Logistic Regression: J&M ch. 4.1-4.4
- Naïve Bayes: J&M ch. B.1-B.6
- Lexicons for Sentiment: J&M ch. 23.1-23.3